

Technical Design Whitepaper

Part Placement Station-Pair Backbone with Extent-Based Diagnostics

Replacing the CM-to-CM child-offset link with an edge-defined, geometry-driven placement model resolved once for simulator and visualizer.

Status	Design / ready to implement — replaces the CM-to-CM child offset (<code>TODO</code>)
Scope	<code>model/parts</code> , <code>model/DesignSerializer</code> , <code>visualizer/RocketMesh</code> — no force-path or integrator changes
Target	QtRocket simulation core (C++23); branch <code>PartPlacement</code>
Audience	Maintainers and future contributors of the simulation core

Abstract

QtRocket currently links each child rocket component to its parent by a single vector: the child's *center of mass relative to the parent's center of mass* (CM-to-CM). Because a part's CM is, in general, *not* at its mid-length — a solid cone's CM sits a quarter of the length from its base — this couples placement to mass distribution, forces authors to hand-compute every offset, and makes the simulator and the 3D visualizer disagree by exactly the part's (geometric-center – CM) offset. The divergence has already produced a silently interpenetrating coupler in a saved design.

This whitepaper specifies the replacement: an **edge-defined, geometry-driven placement model**. Parent and child are related by *physical intent* — a fractional axial station on each part, an axial gap, and a seat kind (abut, nest-in-bore, on-surface) — never an opaque offset. A **single resolver** turns that intent into absolute geometry, and *both* the simulator's composite-inertia pass and the visualizer's mesh walk consume the identical result. The center of mass becomes a purely *derived* quantity. Extent-based diagnostics (a radial seam check, an $O(N \log N)$ envelope sweep with a correct solid-host rule, and a degrees-of-freedom detector) turn the old silent interpenetration into a hard, located failure. A path to 6-DOF is shaped into the data model at zero schema cost but deliberately left unused.

The design is the adjudicated synthesis of five independent sketches, hardened against an adversarial critique. This document renders that synthesis as a buildable specification: UML class diagrams (PlantUML), activity and component diagrams (TikZ), worked numerics against a real test fixture, and citations pinned to the current working tree.

Three non-negotiable rules

1. **Placement is *derived***, recomputed by one resolver — never store absolute coordinates or the resolved pose.
2. **The stored link is *physical intent*** (feature-to-feature), never an opaque offset; CM falls out as a consequence.
3. **The common case costs *zero* authored numbers** — a child with no link abuts its parent.

This document synthesizes [docs/part-placement-design.md](#). Code citations are pinned to the current working tree (branch `PartPlacement`); where the source design spec's line numbers (pinned to an older commit) have drifted, the current locations are used and noted.

Contents

Abstract	ii
1 Introduction & the problem this replaces	1
1.1 How parts are linked today	1
1.2 Why a CM-to-CM link is fragile	2
1.3 The silent divergence between simulator and visualizer	2
1.4 The fix, in one paragraph	3
1.5 Provenance of this specification	3
1.6 Document roadmap	3
2 Design philosophy & the longitudinal convention	4
2.1 The three non-negotiable rules	4
2.2 Settling the longitudinal convention	5
2.3 The per-part-type local frame	5
2.4 The one frame collision: the cone’s internal CM frame	6
3 Core data model: the exact C++ types	7
3.1 The station-pair link	7
3.2 The resolved landmark	9
3.3 The extent profile and the solid-host rule	10
3.4 Degenerate-geometry guards	11
3.5 The explicit cone CM-frame adapter	11
3.6 The storage swap	12
3.7 What lives on Part versus in Placement.h	12
4 The single resolver	15
4.1 Pose and Placed	16
4.2 The resolver signature and per-child algorithm	16
4.3 Why radial placement is a check, not a coordinate	17
4.4 How the center of mass becomes derived	19
4.5 Caching: the resolve must not run per ODE step	20
5 Authoring API & the zero-config default	20
5.1 The primary call and the transitional shim	21
5.2 The zero-config default	21
5.3 Snap-operator verbs	23
6 Overlap & constraint diagnostics	23
6.1 The verdict structures	24
6.2 Layer 1 — radial seam check, on every resolve	24
6.3 Layer 2 — envelope sweep, after the whole tree resolves	26
6.4 Layer 3 — DOF accounting	27
6.5 The gate	28

7	Serialization to .qrd	28
7.1	The <link> element, kept inline	28
7.2	Writer: writeOffset becomes writeLink	29
7.3	Reader: buildPart consumes the link	30
7.4	Version: writer emits 0.2; no loader gate change	30
8	Migration plan	31
8.1	Phase 0 — the invariance gate, written first	31
8.2	Phase 1 — types and resolver land, both APIs coexist	31
8.2.1	The CM-to-CM recovery formula	32
8.3	Phase 2 — run the gate against the migrated reader	33
8.4	Phase 3 — cut over the fixtures and retire the shim	33
9	The 6-DOF path	34
9.1	Orientation already lives in the value types	34
9.2	The one new composition line: rotating the inertia tensor	36
9.3	Off-axis radial seating arrives with rotation, not before	37
9.4	A body-frame integrator reuses the same resolver	37
10	Worked example: xl75_multi	38
11	Deliberately deferred / not adopted	41
12	Files this design touches	43
A	Reproducing this document (toolchain)	46
A.1	Directory layout	46
A.2	The rendering pipeline	47
A.3	Authoring format: a sample class diagram	47
A.4	Commands	48
A.5	Prerequisites	48
B	Glossary	49

1 Introduction & the problem this replaces

QtRocket is an open-source model-rocket simulator built on a shared C++23 simulation core that drives both a Qt6 Widgets GUI and a headless command-line REPL, with a standalone OpenGL viewer sharing the same rocket model. This document specifies a replacement for the way that core expresses *where one part sits relative to another*. The replacement is structural rather than cosmetic: it removes a coordinate convention that is correct only by accident and replaces it with one that is correct by construction.

1.1 How parts are linked today

A rocket is a composite tree of **Part** objects (concrete types such as **ConicalNoseCone**, **BodyTube**, **FinSet**, and **HollowSphere** live under `model/parts/`). Each node stores its direct children together with a single relative-position vector. The storage is, at `model/parts/Part.h:326`,

```
std::vector<std::tuple<std::shared_ptr<Part>, Vector3>> childParts;
```

and the doc comment on that member states the contract precisely: the `Vector3` is “the relative position of the child part’s center of mass with respect to the center of mass of this part.” In other words, parts are linked *center-of-mass to center-of-mass* (CM-to-CM). That single offset is the only spatial relationship the model records; everything downstream — the composite mass properties consumed by the integrator and the geometry drawn by the viewer — is derived from it.

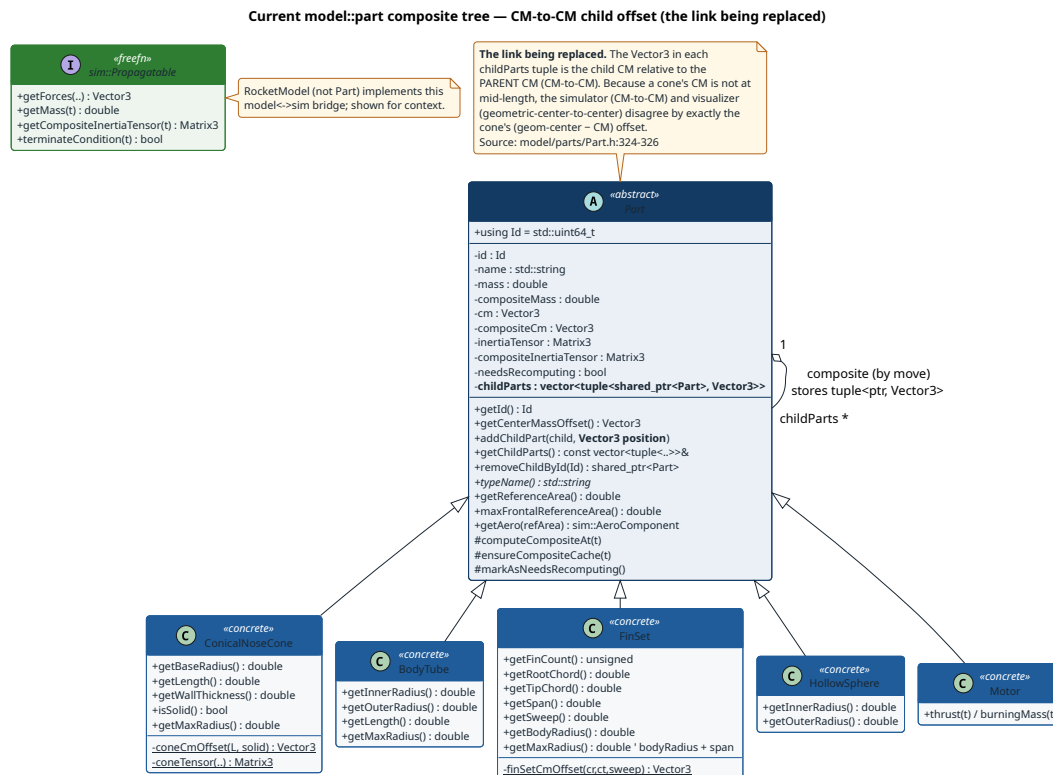


Figure 1. The current composite tree links children CM-to-CM: each edge carries a single `Vector3` measured from parent CM to child CM, and both the simulator and the visualizer reconstruct absolute position by accumulating those offsets.

1.2 Why a CM-to-CM link is fragile

The CM-to-CM link rests on a hidden assumption: that a part’s center of mass is a stable, intuitive landmark an author can reason about. For a longitudinally symmetric part it happens to be — the CM is at mid-length — but [Fig. 1](#) hides the fact that *this is not true in general*. A solid right cone has its centroid one quarter of its length from the base; a thin conical shell has its centroid one third of its length from the base. QtRocket encodes exactly this in `model/parts/ConicalNoseCone.cpp:49`:

```
// With the cone in [-L/2, +L/2] (base at +L/2), the centroid is hbar forward of the base:
// L/4 (solid), L/3 (shell) => z = +L/2 - hbar.
Vector3 ConicalNoseCone::coneCmOffset(double L, bool solid)
{
    const double hbar = solid ? (L / 4.0) : (L / 3.0);
    return Vector3{0.0, 0.0, L / 2.0 - hbar};
}
```

so the cone’s CM is displaced from its geometric center (mid-length) by $L/2 - \bar{h} = L/4$ for a solid cone and $L/6$ for a shell. An author who wants a nose cone’s base rim to meet a body tube’s fore rim must therefore not state that physical intent directly; instead they must hand-compute the offset between two centers of mass that sit at different fractions of their respective lengths. The link records a derived quantity (a CM displacement) where the author actually knows a geometric fact (two rims touch). Every such offset is a manual calculation, and every manual calculation is a place to be wrong.

Key correctness detail

A CM-to-CM link is only as trustworthy as the assumption that CM coincides with the geometric center. That assumption holds for every symmetric part and fails for exactly the part most rockets begin with — the nose cone.

1.3 The silent divergence between simulator and visualizer

The deeper failure is that the two consumers of the link do not even agree on what the offset *means*. The simulation core treats it as documented: in `model/parts/Part.cpp:169`, `Part::computeCompositeAt` accumulates each child’s contribution at `pos + cc.cm` — the stored offset *plus* the child subtree’s own CM — and then shifts every tensor to the composite CM via the parallel-axis theorem. This is a strictly CM-to-CM computation.

The visualizer makes a different assumption entirely. In `visualizer/RocketMesh.cpp`, the `RocketMesh::walk` recursion (the lambda at line 427) builds each primitive *geometrically centered* — `buildCone` and `buildTube` place their mesh symmetrically about the part’s mid-length — and then translates it by an accumulated `station`:

```
const Vector3& offset = std::get<1>(childPair); // the stored CM-to-CM Vector3
if(child)
{
    self(self, *child, station + offset); // accumulated as geometric-center-to-center
}
```

The walk’s own comment names the discrepancy: “`station` = accumulated CM-to-CM offsets from the root down to this part.” The renderer takes a value *defined* as a CM-to-CM displacement and *applies* it as a geometric-center-to-geometric-center translation. For any symmetric part the two interpretations coincide and nothing is visibly wrong. For a nose cone they diverge by precisely the

cone's (geometric center – CM) offset — $L/4$ for a solid cone — and so the simulator and the viewer silently reconstruct two different rockets from the same file.

Hard, located failure

This is not hypothetical. The fixture `tests/data/designs/xl75_multi.qrd:19` places a coupler at `<offset x="0" y="0" z="0.4899999999999999"/>` relative to its body. Read as a CM-to-CM offset by the simulator and as a geometric translation by the viewer — and with the nose's own CM-vs-center displacement folded in on top — the link produces a coupler that interpenetrates the nose cone. The two consumers disagree, no check fires, and the design loads, simulates, and renders without complaint. **FAIL**

1.4 The fix, in one paragraph

The remedy is to remove the divergence by construction. Inter-part relationships are stated as physical *intent* — “this rim seats against that rim,” “this tube nests inside that bore” — expressed as a pair of fractional axial stations plus a seat kind, never as an opaque offset. A **single resolver** walks the ownership tree and produces an absolute, purely *geometric* placement for every part, and *both* the simulator (`computeCompositeAt`, `accumulateAeroAt`) and the visualizer (`RocketMesh::walk`) consume that one placement. Because there is now exactly one source of absolute position, the two consumers cannot disagree. Center of mass ceases to be an input and becomes a purely *derived* consequence of geometry: the resolver places parts by their edges, and the existing two-pass mass-weighted CM and parallel-axis machinery is re-driven from those geometric stations rather than from author-supplied CM offsets. No force-path or integrator change is required, and a path to full six-degree-of-freedom placement is shaped in at zero schema cost.

1.5 Provenance of this specification

This specification is not a single author's first draft. It is the adjudicated synthesis of **five independent design sketches** — feature-mating, datum-pair, declarative constraint-solver, local-frame scene graph, and joint-as-entity graph — judged on three axes (correctness and robustness; simplicity and migration cost; extensibility and ergonomics) and then hardened against an adversarial critique. The judges converged on the datum/station-pair chassis, upgraded it with the extent-based overlap rigor of the constraint-solver sketch, the `Pose` and snap-operator mechanics of the scene-graph sketch, and the direction-constrained nesting of the joint sketch. The rejected alternatives, and *why* each was rejected, are recorded throughout this document rather than discarded, because the rejections are themselves part of the argument for the chosen design.

1.6 Document roadmap

The remainder of this document develops the design in the order an implementer would build it. [Section 2](#) states the three non-negotiable design rules and settles the longitudinal convention ($+z$ = forward, toward the nose tip), including the one frame collision — the cone's internal CM frame — that must be reconciled explicitly. [Section 3](#) gives the exact C++ value types (`StationLink`, `Station`, `SeatKind`) and the in-slot storage swap that replaces the `(Part, Vector3)` tuple. [Section 4](#) specifies the single resolver, how CM becomes derived, and the caching split that keeps placement out of the per-step mass gate. [Section 5](#) covers the authoring API and the zero-config default; [Section 6](#) the three-layer overlap diagnostics that turn the silent interpenetration above into a hard, located failure. [Section 7](#) updates the `.qrd` schema, [Section 8](#) lays out the physics-invariant phased migration,

and [Section 9](#) shows the six-degree-of-freedom path the design is shaped for. [Section 10](#) works the `xl75_multi` fixture end to end; [Section 11](#) records what was deliberately deferred or not adopted; and [Section 12](#), [Appendix A](#), and [Appendix B](#) enumerate the touched files, the reproduction steps, and the glossary.

2 Design philosophy & the longitudinal convention

Before a single type is introduced, the design fixes the small set of invariants that everything downstream must respect. These are deliberately stated as rules rather than as code, because their value is exactly that they constrain every later decision: the data model ([Section 3](#)), the resolver ([Section 4](#)), the serialization format ([Section 7](#)), and the migration ([Section 8](#)) are all forced to be consistent with them. Two things are settled here: the three non-negotiable rules that govern *how* placement is represented and computed, and the single longitudinal sign convention that makes the geometry unambiguous.

2.1 The three non-negotiable rules

The central defect of the legacy link ([Section 1](#)) is that it conflates two ideas that must be kept apart: the *author's intent* (“this rim seats against that rim”) and a *derived numeric placement* (the resolved axial offset). The legacy `Vector3` stored a derived quantity — a center-of-mass-to-center-of-mass offset — and then asked the author to hand-compute it. The rules below exist to make that class of error structurally impossible.

The three non-negotiable rules

- 1. Placement is *derived*, recomputed by exactly one resolver.** Absolute coordinates are never stored, and the resolved pose is never stored. There is a single function ([Section 4](#)) that turns intent into geometry, and both the simulator and the visualizer read its output.
- 2. The stored link is physical *intent*** — a feature-to-feature relationship between a station on the parent and a station on the child — never an opaque offset. The center of mass falls out as a consequence of the geometry, not as an authored input.
- 3. The common case costs zero authored numbers.** A child attached with no explicit link simply abuts its parent; the trivial nose → body → fins stack requires no coordinates at all ([Section 5](#)).

Rule 1 is not a new discipline imposed from outside; it is the discipline the existing codebase already follows for every other derived quantity. The Barrowman aerodynamic contribution is documented as a “*Pure function of geometry; NOT stored (no staleness, no -Werror field)*” (`model/parts/Part.h:154`), and the center-of-mass-versus-middle offset is explicitly marked “*NOT CURRENTLY CONSUMED*” and recomputed rather than cached (`model/parts/Part.h:310`). Composite mass, CM, and inertia are themselves rebuilt lazily behind a dirty flag rather than persisted. Placement is simply the last quantity to be brought under the same rule.

Storing the resolved transform

The obvious shortcut — resolve once, store the resulting transform, and read it back — was considered and rejected. It reintroduces a staleness class that is the exact *inverse* of today's bug: edit a part's length and its stored transform is silently wrong, instead of (as today) edit an offset and the geometry is silently wrong. Either direction leaves two sources of truth that

can disagree. The design keeps a single source of truth — intent — and re-derives geometry on every read, accepting a recompute cost that the caching split in [Section 4](#) keeps off the per-step hot path.

2.2 Settling the longitudinal convention

A placement model needs an unambiguous axial sign convention: a part’s local origin must sit at a defined plane, and “forward” must mean one direction. The design fixes $+z = \text{forward}$ — the local origin of every part lies on the longitudinal axis at its *aft* plane, so the part occupies $z \in [0, +\text{length}]$ moving forward toward the nose tip.

This is not an arbitrary choice. It is the *cheapest correct* choice, because both the renderer and the existing design files already use it; choosing the opposite would impose a sign-inverting rewrite on both with no offsetting benefit.

- **The visualizer is already $+z = \text{forward}$.** `buildCone` places the cone base at $z_{\text{base}} = -L/2$ and the tip at $z_{\text{tip}} = +L/2$ (`visualizer/RocketMesh.cpp:93`), with the base-cap normal pointing $-z$ (“outward = aft”, `visualizer/RocketMesh.cpp:132`). `buildTube` agrees, naming its $+z$ end “forward” and its $-z$ end “aft” (`visualizer/RocketMesh.cpp:156`). The sphere is built from a $+z$ pole to a $-z$ pole on the same axis (`visualizer/RocketMesh.cpp:294`). Every primitive the renderer already emits treats $+z$ as forward.
- **Legacy .qrd files are already $+z = \text{forward}$.** In the `xl75_multi` fixture, the body is placed at a negative z (extending aft, toward $-z$) and the coupler at a positive z relative to the body (forward, toward the nose). The files encode the same sense the renderer does.

$+z = \text{aft}$

Adopting $+z = \text{aft}$ would require a whole-axis inversion of the renderer — every cone base-cap normal, every sphere pole, every primitive endpoint — *and* a sign-negating migration of every legacy file. Under $+z = \text{forward}$, by contrast, wiring the renderer to the new resolver is a pure *consumer swap* with no sign inversion, and the file migration ([Section 8](#)) is a re-expression of the same numbers with no axial flip. The alternative pays a large, error-prone cost to arrive at an equivalent model.

2.3 The per-part-type local frame

With $+z = \text{forward}$ fixed, each concrete part type’s local frame is fully determined. The aft plane is always the origin; the part extends forward to its length.

- **ConicalNoseCone.** The aft plane (the base rim) is at $z = 0$ with $r = \text{baseRadius}$; the fore point (the tip) is at $z = +\text{length}$ with $r = 0$. The outer radius tapers linearly, $\text{radiusOuterAt}(z) = \text{baseRadius} (1 - z/\text{length})$.
- **BodyTube, FinSet, HollowSphere.** The aft plane is at $z = 0$ and the fore plane at $z = +\text{length}$.

[Figure 2](#) also depicts the one place where the chosen convention does not match a pre-existing frame inside the code, which the next subsection treats as the single most correctness-critical detail of the whole design.

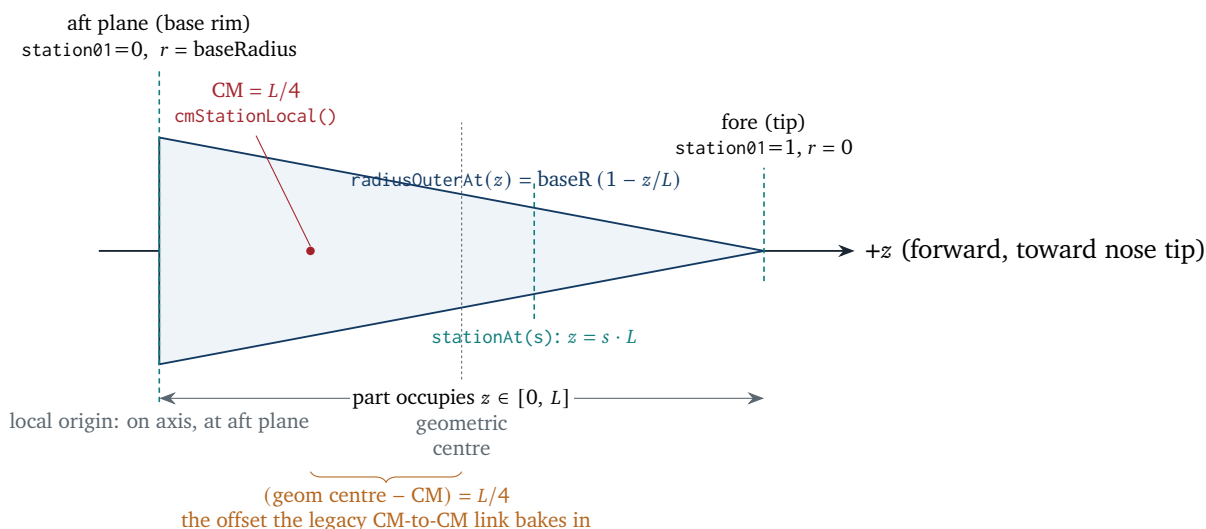


Figure 2. The settled $+z = \text{forward}$ convention. Each part's local origin sits on the longitudinal axis at its aft plane, occupying $z \in [0, +\text{length}]$. The single frame collision — the cone's internal CM frame, whose origin is at mid-length and whose $+z$ points toward the base — is shown opposed to the chosen forward direction and is reconciled by an explicit adapter rather than by altering the cone's tensor math.

2.4 The one frame collision: the cone's internal CM frame

The cone's inertia tensor is centroidal, and the helper that reports where that centroid sits — `coneCmOffset` (`model/parts/ConicalNoseCone.cpp:49`) — works in a frame that is *opposite* to the convention just settled. It places the cone in $[-L/2, +L/2]$ with the base at $+L/2$, so its centroid is computed as $z = +L/2 - \bar{h}$, with $\bar{h} = L/4$ for a solid cone and $L/3$ for a thin shell:

```
// The tensor is centroidal; report where that CM sits relative to the component middle so
// the assembly's addChildPart position is CM-to-CM. With the cone in  $[-L/2, +L/2]$  (base at
//  $+L/2$ ), the centroid is  $\bar{h}$ bar forward of the base:  $L/4$  (solid),  $L/3$  (shell)  $\Rightarrow z = +L/2 - \bar{h}$ bar.
Vector3 ConicalNoseCone::coneCmOffset(double L, bool solid)
{
    const double hbar = solid ? (L / 4.0) : (L / 3.0);
    return Vector3{0.0, 0.0, L / 2.0 - hbar};
}
```

Listing 1. The cone's internal CM frame (`model/parts/ConicalNoseCone.cpp:49`): mid-length origin, base at $+z$ — the reverse of the chosen $+z = \text{forward}$ placement frame, whose origin is the aft (base) plane with the tip at $+z$.

This frame (Listing 1) is the only place in the codebase whose $+z$ disagrees with the chosen forward direction. The tempting fix — flip the cone's tensor and CM math so its $+z$ matches — is precisely the wrong move: that math is correct, is exercised by the existing inertia tests, and is unrelated to placement. Inverting it to satisfy a placement convention would risk a correctness regression in the physics core in order to avoid a two-line coordinate transform.

The single most correctness-critical detail

The cone's internal CM frame (mid-length origin, base at $+z$) is the reverse of the chosen

$+z$ = forward placement frame (aft-plane origin, tip at $+z$). The design reconciles the two with *one explicitly specified, unit-tested adapter* — not by flipping the cone’s tested tensor math. In the placement frame the cone’s CM lies at $\text{cmZ}_{\text{fwd}} = \bar{h}$, i.e. $L/4$ forward of the base for a solid cone and $L/3$ for a shell, equivalently $L - \bar{h}$ aft of the tip ($3L/4$ for a solid cone). This adapter (developed in [Section 3](#)) is the *only* consumer of `getCenterMassOffset()` (`model/parts/Part.h:96`); it lives behind the new `cmStationLocal()` virtual, which the cone is the only part type to override for a frame flip. For a part whose CM is at mid-length $\text{cmZ}_{\text{fwd}} = L/2$. Getting this one transform right is what makes the cone’s CM a correctly derived consequence of the geometry rather than a quietly inverted offset.

Isolating the collision to a single adapter is what allows the rest of the design to treat placement as pure geometry. No implicit cross-frame vector addition survives anywhere else: the resolver works entirely in the $+z$ = forward placement frame, and the only translation between frames happens at one named, tested boundary.

3 Core data model: the exact C++ types

The placement design lives or dies on the precision of a handful of value types and virtual functions. This section fixes them exactly: the stored *intent*, the derived *landmarks*, the geometric profile that powers diagnostics, the one cross-frame adapter that keeps the cone honest, and the single in-slot storage change that carries it all. The guiding principle established in [Section 2](#) is enforced here at the type level: the only thing a part stores about its children is physical intent; every absolute coordinate, every center of mass, and every overlap verdict is *derived* by the resolver of [Section 4](#) from these types.

A note on the starting point, because it governs how the rest of this section reads. The working tree is entirely pre-refactor. The new types named below — `StationLink`, `SeatKind`, `Station`, `Pose`, and the resolver — do not yet exist, and the file `model/parts/Placement.h` that will house the free types is new. The virtuals `stationAt`, `radiusOuterAt`, `radiusInnerAt`, `axialLength`, `isSolid`, and `cmStationLocal` are *not* on `Part` today; the only geometry-adjacent virtual the base class currently exposes is `model/parts/Part.h:176` (`getReferenceArea()`, default `0.0`). The discussion below therefore describes the target types and, where it matters for correctness, flags precisely what must be added versus what already exists.

[Figure 3](#) shows the new value types of `model/parts/Placement.h` and how the free-function resolver consumes them; the companion [Fig. 4](#) (landscape, below) shows the target `Part` after the geometry virtuals are added.

3.1 The station-pair link

A parent–child relationship in a rigid axisymmetric stack reduces to remarkably little information: a fractional axial station on the parent, a fractional axial station on the child, an axial gap, and a seat kind that selects the radial compatibility check and fixes the sign of the gap. That is the whole of `StationLink`. Stations are stored as fractions of *live* length and resolved against the part’s length on every read, so editing a part’s length moves the seam automatically rather than silently invalidating a cached number — the staleness class this design exists to eliminate.

The two members `parentStation01` and `childStation01` are dimensionless fractions in $[0, 1]$, with 0 the aft plane and 1 the fore plane in the $+z$ = forward convention. The `gap` is in meters and its meaning is selected by `seat`. The `childRot` member is the per-seam orientation reserved for

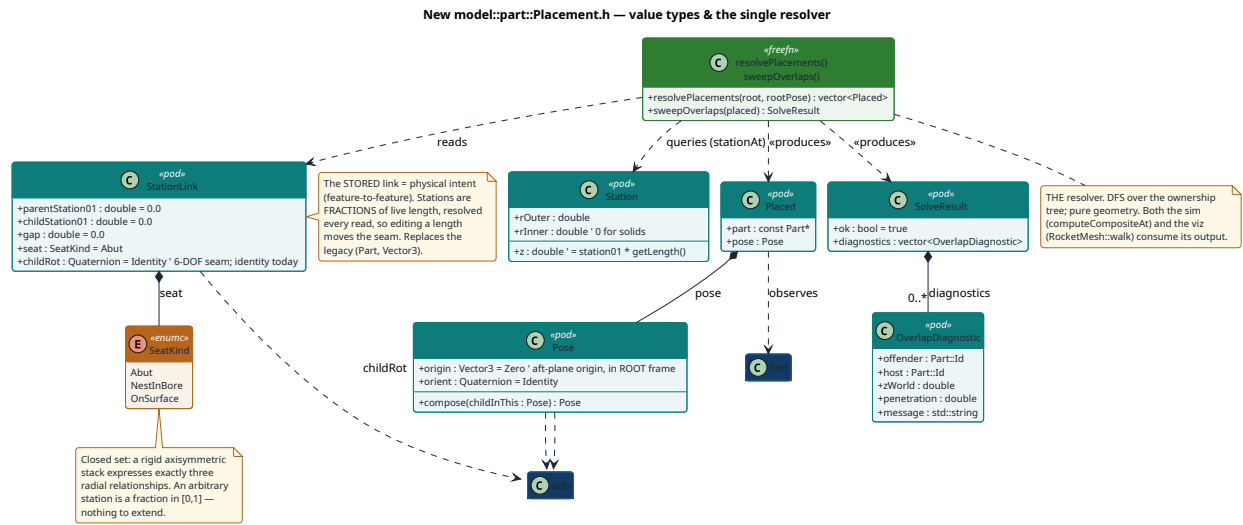


Figure 3. New Placement.h value types and the resolver.

```
namespace model::part {

// How a child seats against its parent. Governs (a) the radial compatibility CHECK
// and (b) the SIGN of the gap (insertion direction).
enum class SeatKind : std::uint8_t {
    Abut,          // rim-to-rim, same radius; gap is a forward standoff (signed +z)
    NestInBore,    // child OD seats inside parent ID; gap is insertion DEPTH
                  // (child moves AFT, -z, into the bore)
    OnSurface      // child seats radially on parent's outer wall at the station;
                  // gap is an axial standoff
};

} // namespace model::part
```

Listing 2. The SeatKind enum: three radial relationships, governing the radial check and the gap sign.

```
// Physical intent for one parent->child attachment. Stations are FRACTIONS of live
// length, resolved against getLength() every read, so editing a length moves the seam.
struct StationLink {
    double parentStation01{0.0}; // 0 = aft plane, 1 = fore plane, on the PARENT
    double childStation01 {1.0}; // 0 = aft plane, 1 = fore plane, on the CHILD (default: fore)
    double gap{0.0};             // metres; meaning per SeatKind (standoff or depth)
    SeatKind seat{SeatKind::Abut};
    Quaternion childRot{Quaternion::Identity()}; // 6-DOF seam; identity today
};
```

Listing 3. The StationLink struct: two fractional stations, an axial gap, a SeatKind, and a (currently identity) seam rotation.

6-DOF (Section 9); it is `Quaternion::Identity()` today, so the present behavior is bit-identical to a pure-translation tree, and it costs no schema reshape to activate later.

A rich named-datum enum

An alternative design enumerated named attachment data — `FORE`, `MID`, `AFT`, and so on — with a parametric escape hatch for everything in between. It was rejected as paying both costs at once: it is *too rigid* (a rail button partway up a tube has no named datum and forces the escape hatch immediately) and *too rich* (the named edges `FORE/MID/AFT` are nothing more than the fractions 1, 0.5, and 0). Two fractional stations in $[0, 1]$ subsume the named edges, the arbitrary mid-body station, and the escape hatch in a single uniform representation, with nothing left to extend.

`SeatKind`, by contrast, stays deliberately closed. A rigid axisymmetric stack genuinely expresses only three radial relationships: rim-to-rim of equal radius (**Abut**), one part's outer diameter seated inside another's inner diameter (**NestInBore**), and a part seated radially on another's outer wall (**OnSurface**). The enum's two jobs are (a) to choose the radial compatibility check applied at the seam — equal radii for **Abut**, offender outer radius against host capacity for **NestInBore**, wall match for **OnSurface** — and (b) to fix the sign of the gap. In the $+z$ = forward frame, **Abut** and **OnSurface** treat the gap as a forward standoff ($+z$), whereas **NestInBore** treats it as a non-negative insertion depth that moves the child *aft* ($-z$) into the bore. There is no fourth radial relationship to add for the parts QtRocket models, so leaving room to extend would only invite half-defined cases.

3.2 The resolved landmark

Given a fractional station, a part must report a concrete geometric landmark in its own $+z$ = forward local frame: the axial coordinate and the outer and inner radii at that station. That is the `Station` struct, the geometry-derived value that is computed on demand and *never* stored.

```
struct Station {
    double z{0.0}; // axial station, part-local +z-forward (m) = station01 * getLength()
    double rOuter{0.0}; // outer radius at z (m)
    double rInner{0.0}; // inner radius at z (m); 0 for solids
};

// On Part. Base implementation uses getLength() + the radius virtuals below, so
// symmetric parts need NO override. Clamps station01 to [0,1] (see degenerate guards).
virtual Station stationAt(double station01) const;
```

Listing 4. The derived `Station` landmark and the base `stationAt` virtual.

The base implementation of `stationAt` is written once on `Part` in terms of the part's length and the radius-profile virtuals of Section 3.3: it maps the fraction to an axial coordinate, $z = \text{station01} * \text{getLength}()$, and queries the outer and inner radius at that coordinate. Because the entire computation is expressed through those virtuals, every symmetric part inherits a correct `stationAt` with no override at all. A part overrides it only if its station-to-radius mapping is not captured by the default profile.

This is where the design must add machinery the base class does not yet have.

getLength() is promoted to a base virtual

There is currently *no* `getLength()` on `Part`. Each concrete part carries its own non-virtual accessor — `BodyTube` at `model/parts/BodyTube.h:51` and `ConicalNoseCone` at `model/parts/ConicalNoseCone.h:53` — and `HollowSphere` (`model/parts/HollowSphere.h`) has none at all, because a sphere has no length member. The design *promotes* `getLength()` to a base virtual so the shared `stationAt` can call it polymorphically. `HollowSphere` must then supply a definition (its axial extent is the diameter, $2r_{\text{outer}}$), and the existing per-part accessors become overrides. This is added work, not a pre-existing facility.

3.3 The extent profile and the solid-host rule

Diagnostics need more than a single landmark; they need the part’s radial profile along its whole axial span. Three closed-form virtuals supply it, each defaulting to a geometrically inert part on the base class so a part with no real cross-section needs no override.

```
// On Part. Closed-form per part type. Base returns 0 (geometrically inert).
virtual double radiusOuterAt(double zLocal) const { return 0.0; } // cone tapers; tube const
virtual double radiusInnerAt(double zLocal) const { return 0.0; } // BodyTube bore; 0 solids
virtual double axialLength() const { return getLength(); } // span is [0, axialLength()]
```

Listing 5. The extent-profile virtuals: outer radius, inner radius, and axial span, in the $+z$ = forward local frame.

The reason these are distinct from `stationAt` is that overlap detection samples the profile at many stations across a span (Section 6), so the per- z radius functions must be callable independently. `radiusOuterAt` returns the tapering skin for a cone, the constant outer wall for a tube, and the bulging silhouette for a sphere; `radiusInnerAt` returns the bore radius for a hollow part and zero for a solid. `axialLength` defaults to the promoted `getLength()`, which is correct for the longitudinal parts and is the natural override point for `HollowSphere` (whose span is its diameter).

The subtle correctness problem is deciding, for a given host part and station, the radial budget an intruding part is allowed to occupy. A naive “offender outer radius versus host *inner* radius” test is wrong for a solid cone: a solid cone has no bore, so `radiusInnerAt` returns zero everywhere, and the naive test would flag *every* intruder as poking through. The fix is a single predicate that selects the right boundary by whether the host is bored or solid.

The solid-host rule

A host occupies radius $[0, \text{hostCapacity}(z)]$, where

$$\text{hostCapacity}(z) = \begin{cases} \text{radiusInnerAt}(z) & \text{for a bored part (the bore),} \\ \text{radiusOuterAt}(z) & \text{for a solid part (the outer skin).} \end{cases}$$

An offender at station z with outer radius r_{off} fits if and only if $r_{\text{off}} \leq \text{hostCapacity}(z) + \text{tol}$.

This one predicate is correct for both cases. For a bored tube the offender must fit inside the bore (`radiusInnerAt`); for a solid cone it must fit inside the outer skin (`radiusOuterAt`) — which is exactly the poke-through test, the geometric question of whether the intruder breaks the cone’s outer surface. The rule is exposed as a single non-virtual helper,

`double` `Part::innerCapacityAt(double zLocal) const`, so call sites in the overlap sweep never branch on solidity themselves.

The branch is driven by one more virtual:

```
virtual bool isSolid() const { return radiusInnerAt(0.0) <= 0.0; }
```

The base default infers solidity from the absence of a bore, which is correct for a solid cone. Parts with a wall — `BodyTube` (whose annular wall makes the bore explicit, members at `model/parts/BodyTube.h:67`) and `HollowSphere` (a shell, members at `model/parts/HollowSphere.h:89`) — override `isSolid` to report their wall honestly, so `innerCapacityAt` routes through the bore for them.

3.4 Degenerate-geometry guards

The profile functions must be total: a malformed or zero-extent part is a load-time diagnostic, not undefined behavior or a divide-by-zero. Three guards make that so.

- `stationAt` clamps `station01` to $[0, 1]$. A link whose fraction falls outside the range is reported as a load-time warning rather than producing an out-of-range station.
- `radiusOuterAt` for the cone guards the taper divide: when $\text{length} \leq 10^{-9}$ it returns `baseRadius` (a degenerate zero-length disc or ring) instead of evaluating $1 - z/\text{length}$ as $0/0$.
- A zero-length part (a ring or transition) has `axialLength() = 0`, so `stationAt(any)  $\rightarrow z = 0$` . Its overlap span collapses to a single station, which the envelope sweep of [Section 6](#) handles as a point sample. The case is documented behavior, not a crash.

3.5 The explicit cone CM-frame adapter

The single most correctness-critical detail in the data model is reconciling the cone’s internal center-of-mass frame with the resolver’s frame. The cone computes its centroidal inertia and its CM offset in a *mid-length-origin, base-at-+z* frame whose +z points opposite to the chosen forward direction. That math is correct and unit-tested, so the design does not flip it; instead it applies one explicitly specified adapter at exactly one place.

The accessor that exposes the cone’s CM offset is `model/parts/Part.h:96` (`Vector3 getCenterMassOffset() const { return cm; }`, returning the part’s private `cm` member). For the cone, `cm` is set at construction from `coneCmOffset` (`model/parts/ConicalNoseCone.cpp:49`), which returns $\{0, 0, L/2 - \bar{h}\}$ with $\bar{h} = L/4$ for a solid cone and $\bar{h} = L/3$ for a shell, measured in that mid-length-origin frame where the base sits at $+L/2$.

The resolver, by contrast, works in the aft-plane-origin, $+z = \text{forward}$ frame, where the cone base is at $z = 0$ and the tip at $z = L$. The transform between the two frames is fixed and stated once:

The cone CM in the forward frame

For the cone, the CM’s forward-frame axial coordinate is

$$\text{cmZ_fwd} = \bar{h},$$

i.e. $L/4$ forward of the base for a solid cone ($\bar{h} = L/4$) and $L/3$ forward of the base for a shell ($\bar{h} = L/3$). Equivalently, the CM sits $L - \bar{h}$ aft of the tip ($3L/4$ aft of the tip for a solid cone). This follows from the internal datum: `getCenterMassOffset()` reports $z_{\text{int}} = L/2 - \bar{h}$ in the cone’s mid-length, base-at-+z frame, and the forward-frame coordinate (base at $z = 0$) is $L/2 - z_{\text{int}} = \bar{h}$. The *wide base* carries the mass, so the CM sits near the base, not the tip — a sign the adapter must get right. For a part whose CM is at mid-length, `cmZ_fwd` = $L/2$. The resolver obtains each child’s

own-frame CM as a single scalar through a small per-type adapter `Part::cmStationLocal()`; the cone is the only part that overrides it for a *frame flip*, and `getCenterMassOffset()` is consumed only inside `cmStationLocal()` — never as a link, never anywhere else.

This is what finally puts `getCenterMassOffset()` to its documented purpose. The base `cmStationLocal()` returns $L/2$ from the promoted `getLength()`; `ConicalNoseCone` overrides it to return $\bar{h}$, translating the cone’s internal CM datum into the forward frame. `BodyTube` (`model/parts/BodyTube.cpp:15`) and `HollowSphere` (`model/parts/HollowSphere.cpp:15`) hand the `Part` base a centroidal tensor about a geometric-center CM with no offset, so for them $\text{cmZ}_{\text{fwd}} = L/2$ is exact and the default suffices. `FinSet` (`model/parts/FinSet.cpp:52`) is the subtler case: its axial CM sits at the fin-set mass centroid (a *non-zero* `getCenterMassOffset()`), not at mid-length, so it too must report that offset through `cmStationLocal()` — though, unlike the cone, with no frame flip. With the adapter localized to one method, no implicit cross-frame vector addition survives anywhere in the composition path — the historical source of the simulator/visualizer divergence described in [Section 2](#).

3.6 The storage swap

With the link type defined, the storage change on `Part` is a single in-slot struct swap. Children are stored today at `model/parts/Part.h:326` as `std::vector<std::tuple<std::shared_ptr<Part>, Vector3>>`, the `Vector3` being the child’s center of mass relative to the parent’s center of mass — the CM-to-CM coupling this design replaces. It becomes:

```
std::vector<std::pair<std::shared_ptr<Part>, StationLink>> childParts;
```

Listing 6. The storage swap: the per-child `Vector3` offset becomes a `StationLink` of physical intent, kept in a single vector.

Keeping a *single* vector is a deliberate choice.

Parallel `childParts` and `childLinks` arrays

Splitting the pointers and the links into two parallel vectors was rejected as a desynchronization hazard. Every operation that reshapes the child list — `clone()`, `removeChildById()`, and `addChildPart()` — would have to keep two arrays index-aligned, and any future edit that touches one but not the other silently corrupts the mapping. A single vector of `pair<ptr, StationLink>` makes the pointer and its intent inseparable by construction; there is no index to keep aligned.

The swap is in-slot: the storage stays one vector of pairs, the same accessor pattern reads it, and the composition routines that destructure it ([Section 4](#)) change only their second binding. Because the second element of the pair is no longer consumed as a coordinate but as intent fed to the resolver, the `<tuple>` dependency the current header carries solely for this member can be dropped once the swap lands.

3.7 What lives on `Part` versus in `Placement.h`

The design draws a clean line between behavior that must be polymorphic on a part — the geometry virtuals — and the plain value types and free functions that operate over a resolved tree. [Table 1](#) states the split.

Table 1. Where each new construct lives: behavior on `Part`; values and resolver in the new `model/parts/Placement.h`.

Lives on <code>Part</code>	Lives in <code>Placement.h</code> (free / POD)
<code>stationAt</code> , <code>radiusOuterAt/radiusInnerAt</code> , <code>axialLength</code> , <code>isSolid</code> , <code>cmStationLocal</code> (virtuals); promoted <code>getLength()</code> virtual	<code>Station</code> , <code>StationLink</code> , <code>SeatKind</code> , <code>Pose</code>
<code>childParts</code> (now <code>pair<ptr, StationLink></code>); non- virtual helper <code>innerCapacityAt</code>	<code>resolvePlacements(...)</code> , <code>sweepOverlaps(...)</code>
<code>addChildPart(child, StationLink)</code> plus a transi- tional <code>Vector3</code> overload	<code>OverlapDiagnostic</code> , <code>SolveResult</code>

The rationale for the division is that the virtuals encode *per-part geometry* and must dispatch on dynamic type, so they belong on `Part`; the value types are pure data with no behavior, and the resolver and sweep are pure functions over a whole tree, so they belong in a free header that both the model and the visualizer can include without dragging in the part hierarchy. The transitional `Vector3` overload of `addChildPart` (detailed in [Section 5](#) and [Section 8](#)) coexists with the `StationLink` primary during migration so the existing call sites and `.qrd` fixtures keep compiling; the present tree has exactly one overload, the legacy `addChildPart(std::shared_ptr<Part>, Vector3)` at `model/parts/Part.h:239`.

Target model::part::Part — station landmarks, StationLink storage, derived CM

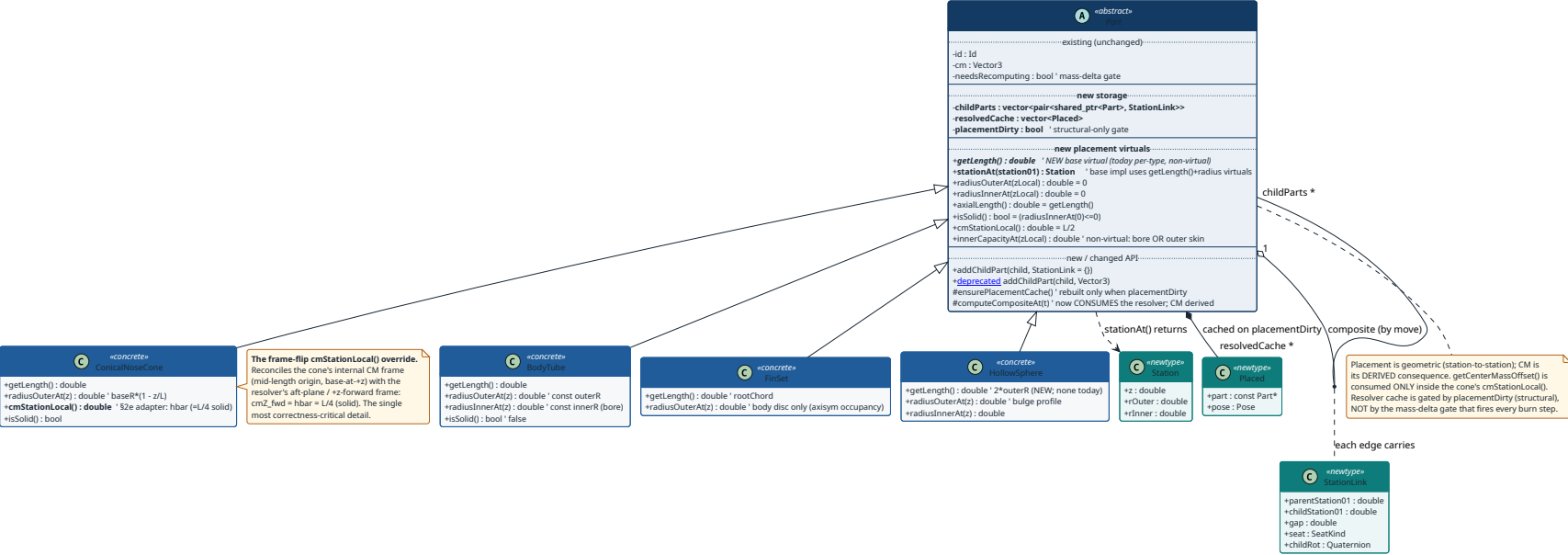


Figure 4. Target Part after the refactor.

Figure 4 shows the target `Part` once these virtuals, the storage swap, and the authoring overload are in place. The composition routines that consume the resolved placement, and the resolver itself, are the subject of Section 4.

4 The single resolver

The data model of Section 3 stores only physical *intent*: a child carries a `StationLink` describing which feature of itself seats against which feature of its parent. Nothing in that model is an absolute coordinate. The job of turning a tree of intents into a set of absolute placements — one transform per part, expressed in a common root frame — belongs to a *single* pure function. This section specifies that function, shows how the center of mass becomes a quantity *derived* from its output rather than an authored input, and explains the caching discipline that keeps the geometry resolve off the per-step integration hot path.

The architectural commitment is that there is exactly one authority for absolute placement, and that both the simulator’s composite-mass code and the visualizer’s mesh walker consume *its* output rather than computing their own. Figure 5 shows the resulting shape: the resolver sits between the part tree and its two consumers, and the divergence described in Section 1 — in which `model/parts/Part.cpp:224` threaded a CM-to-CM axial offset while `visualizer/RocketMesh.cpp:472` threaded a geometric-center offset — is removed by construction, because neither consumer is permitted to invent its own placement.

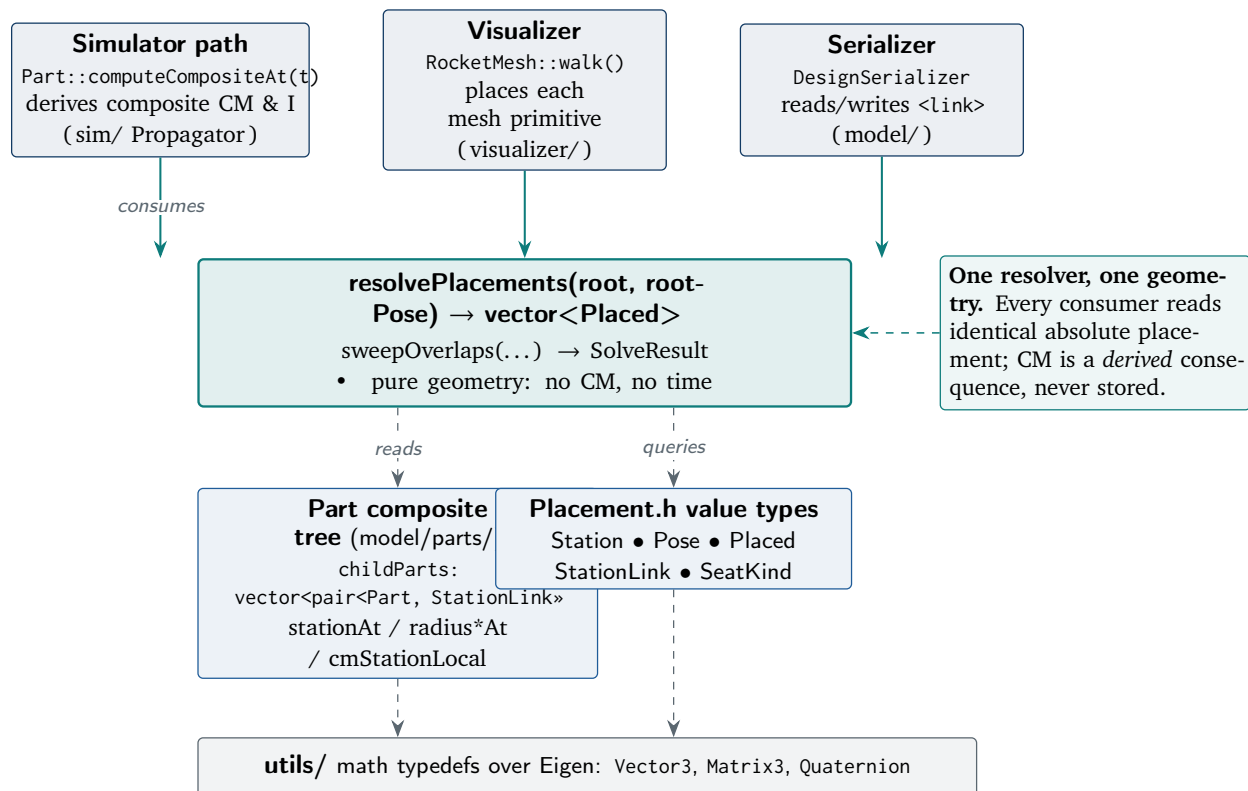


Figure 5. One resolver is the sole authority for absolute placement; the simulator’s composite/aero code and the visualizer’s mesh walk are both pure consumers of its output.

4.1 Pose and Placed

The resolver works in terms of a **Pose**: the rigid transform that carries a part’s local frame (origin on the longitudinal axis at the part’s aft plane, $+z$ = forward) into the shared root frame. A **Pose** is an origin translation plus an orientation quaternion, and the only operation it needs is composition — placing a child whose pose is known *relative to its parent* into the root frame by left-multiplying the parent’s pose (Listing 7).

```
struct Pose {
    Vector3    origin {Vector3::Zero()};          // aft-plane origin, on axis, in ROOT frame
    Quaternion orient {Quaternion::Identity()};    // (x,y,z,w); identity in 3-DOF

    Pose compose(const Pose& childInThis) const {
        return Pose{ origin + orient * childInThis.origin,
                     (orient * childInThis.orient).normalized() };
    }
};

struct Placed { const Part* part; Pose pose; }; // deterministic DFS (attachment) order
```

Listing 7. Pose and Placed (new, in `model/parts/Placement.h`). `compose` rotates the child translation by the parent orientation before adding it, so the same code is exact for the present pure-translation tree (`orient` = identity) and for the future 6-DOF tree without reshape.

Writing composition this way — rotate-then-translate, with the quaternion product renormalized — is deliberate. In the present 3-DOF regime every `orient` is the identity, so `compose` degenerates to vector addition and the resolved tree is bit-identical to the legacy pure-translation walk. But the *same line* is correct once orientations become non-trivial (Section 9): the child’s local origin is rotated into the parent frame before the parent’s translation is applied, which is exactly the rigid-transform composition law. Carrying the quaternion from day one costs one identity multiply per child today and buys a zero-schema-change path to 6-DOF.

A **Placed** is simply a non-owning pointer to a part paired with its resolved pose. The resolver returns these in a deterministic depth-first, attachment-order sequence, which both gives the overlap sweep of Section 6 a stable ordering and makes the resolved sequence reproducible across runs.

4.2 The resolver signature and per-child algorithm

```
// THE resolver. DFS over the ownership tree from rootPose.
// Pure geometry: no center of mass, no time, no mass.
std::vector<Placed>
resolvePlacements(const part::Part& root, const Pose& rootPose);
```

Listing 8. The single resolver entry point (new, in `model/parts/Placement.h`). It takes the tree root and the pose at which to plant it (conventionally identity — the rocket’s aft datum at the world origin, $+z$ = forward) and returns one **Placed** per part.

The resolver visits the root at `rootPose` and then descends. For each child of an already-placed parent, carrying `StationLink m`, the placement is computed entirely from the two parts’ *geometry* — never from any stored coordinate (Listing 9). The parent and child are each asked for the live

landmark at the relevant fractional station via `stationAt` (Section 3); the seat kind then fixes both the radial compatibility check and the *sign* of the gap.

```
// parentPose is already known (root planted at rootPose; children by DFS).
const Station p = parent.stationAt(m.parentStation01); // parent frame
const Station c = child .stationAt(m.childStation01);  // child frame

// signedGap encodes the seat direction in +z = FORWARD:
// Abut      => +m.gap  (forward standoff; flush at gap = 0)
// OnSurface => +m.gap  (axial standoff along the parent wall)
// NestInBore => -m.gap  (m.gap = insertion depth >= 0; child seats AFT, -z, into bore)
const double signedGap = (m.seat == SeatKind::NestInBore) ? -m.gap : +m.gap;

// Child's aft-plane origin along the parent axis: line the two stations up,
// then displace by the signed gap.
const double childOriginZ = p.z + signedGap - c.z;

// Coaxial today: x = y = 0. (Radial r is a CHECK, not a placement -- see below.)
const Pose childInParent{ Vector3(0.0, 0.0, childOriginZ),
                          m.childRot /* Identity in 3-DOF */ };
const Pose childInRoot = parentPose.compose(childInParent);
```

Listing 9. The per-child placement step, in $+z = \text{forward}$. Pure geometry: the child’s aft-plane origin is placed so the chosen child station lands on the chosen parent station, offset by the seat-signed gap; the result is composed into the root frame.

The expression `childOriginZ = p.z + signedGap - c.z` reads directly as physical intent. The parent station sits at axial coordinate `p.z` in the parent frame; the child must be positioned so that its own chosen station `c.z` lands there (hence the `- c.z`, because the pose locates the child’s *aft plane*, not its station); and the seat-signed gap then separates or inserts the two. The seat kind is what makes the sign correct without authored negatives: **Abut** and **OnSurface** treat the gap as a forward standoff, so a positive gap moves the child forward ($+z = \text{forward}$); **NestInBore** treats the gap as an insertion *depth*, so a positive depth drives the child aft into the bore. Encoding the direction in the seat kind, rather than asking the author to sign the gap, is what lets a **NestInBore** of depth 0.04 and an **Abut** standoff of 0.04 both be written as a plain positive number.

Figure 6 traces the full control flow: plant the root, then for each child in DFS order sample both stations, run the Layer-1 radial seam check (Section 6), compute `childOriginZ`, build the local pose, compose into the root frame, and recurse.

4.3 Why radial placement is a check, not a coordinate

The per-child step sets $x = y = 0$: every child is placed coaxially with its parent. The radial value r recovered from the two stations (`p.rOuter`, `p.rInner`, `c.rOuter`) is used *only* for the seam and overlap *check* of Section 6 — never to displace the pose off-axis.

This is a deliberate scope line, not an oversight. It is exactly correct for every part type QtRocket models today: tubes, cones, and spheres are axisymmetric, and a fin set’s mass distribution is on-axis by symmetry, so their physically correct 3-DOF placement *is* coaxial. The temptation is to support a genuinely off-axis feature — a single rail button, an asymmetric pod — by setting $x = r$ in the pose. We reject that in 3-DOF on correctness grounds.

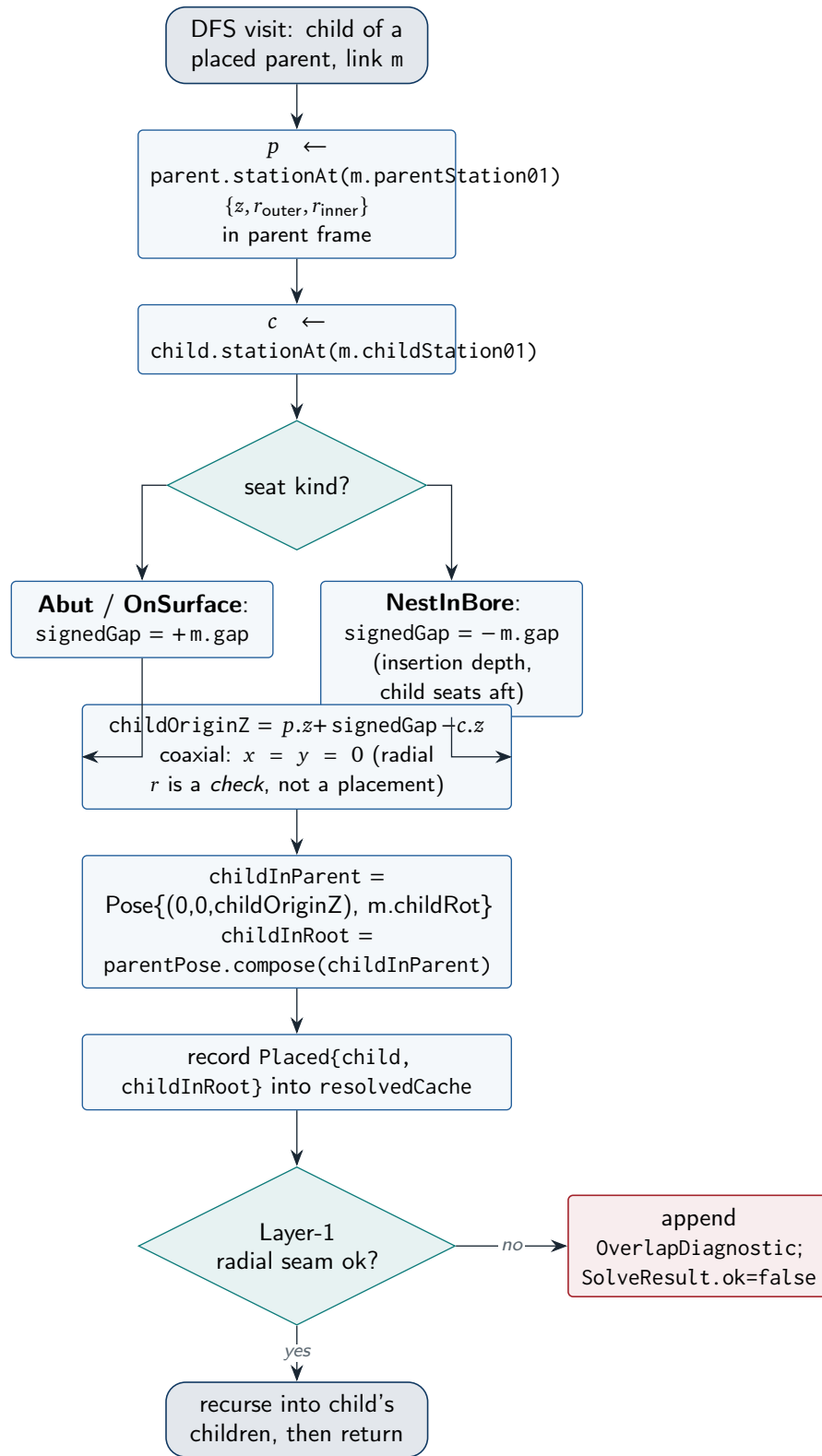


Figure 6. The resolver's depth-first traversal: each child is placed purely from the two parts' live station landmarks and the seat-signed gap, then composed into the root frame.

Off-axis seating in 3-DOF

Placing a part at $x = r$ without also rotating it into a body-frame orientation stores a *half-correct* placement: the mass is moved off the axis, but the part is not oriented to sit against the surface it nominally seats on, and no per-part body-frame CM offset is applied. That is precisely the class of silently-wrong stored placement this entire design exists to eliminate — just relocated from the axial axis to the radial one. True off-axis seating is therefore deferred to the 6-DOF work (Section 9), where `StationLink::childRot` and a per-part body-frame CM offset make an **OnSurface** placement at $x = r$ *correctly oriented*. Until then r earns its keep as a diagnostic, not as a coordinate.

4.4 How the center of mass becomes derived

The legacy composite routine consumed the stored CM-to-CM `Vector3` directly: in `computeCompositeAt` the child's CM in the parent frame was `pos + cc.cm`, where `pos` was the authored offset (the two-pass loop at `model/parts/Part.cpp:179`). The refactor severs that coupling. The placement is now *geometric* (station-to-station, produced by the resolver), and the CM is its derived consequence.

Crucially, the two-pass, mass-weighted CM plus parallel-axis structure of `computeCompositeAt` is *preserved unchanged in shape*. Pass 1 still accumulates a mass-weighted sum to find the composite CM; pass 2 still shifts every child tensor to that CM via the parallel-axis displacement tensor `parallelAxisTerm` at `model/parts/Part.cpp:17` (the helper $f(d) = (d \cdot d) I_3 - d d^T$ used twice in the composite loop). What changes is solely the *input* to that arithmetic. Instead of reading the stored `pos`, each child's CM in the root frame is derived from its resolved pose and its own-frame CM station (Listing 10).

```
// For each child, from its resolved Pose and its OWN-frame CM station:
const Vector3 childCmInRoot =
    childPose.origin
    + childPose.orient * Vector3(0.0, 0.0, child.cmStationLocal());
// (+ child.getCenterMassOffset() radial parts, zero for every current part)

// Pass 1 (mass-weighted CM): weighted += childMass(t) * childCmInRoot
// Pass 2 (parallel-axis):   d = childCmInRoot - compositeCm
//                           I += childI + childMass * parallelAxisTerm(d)
//                           AND (6-DOF) I' = R * I * R^T, R = childPose.orient
```

Listing 10. The CM derivation. `cmStationLocal()` returns the child's CM as a scalar axial station in its own $+z$ =forward frame; it is the only place `getCenterMassOffset()` (`model/parts/Part.h:96`) is consumed. The cone overrides it to apply the explicit frame adapter of Section 3; all symmetric parts return $L/2$.

The single scalar `cmStationLocal()` is what makes this safe across the cone's frame collision. The cone's internal CM helper `coneCmOffset` at `model/parts/ConicalNoseCone.cpp:49` expresses the CM in a mid-length-origin, base-at- $+z$ frame whose forward sense is the *opposite* of the resolver's. Rather than risk an implicit cross-frame vector addition — the original sin this design corrects — the cone is the one part that overrides `cmStationLocal()` to return its CM as a single, unit-tested scalar in the $+z$ =forward frame, and every symmetric part returns $L/2$. No vector arithmetic crosses a frame boundary unwatched.

The aero path is updated in the same spirit. `accumulateAeroAt` previously threaded the axial station as `axialStation + pos.z()` in the recursive descent at `model/parts/Part.cpp:224`. It now threads each child's resolved `childPose.origin.z()` — the same root-frame station every other

consumer sees. The visualizer’s `walk` lambda, which recursed with `station + offset` at `visualizer/RocketMesh.cpp:472` and read the legacy tuple at `visualizer/RocketMesh.cpp:468`, instead translates each geometric-centered primitive by the resolver’s `Pose.origin` (and, when 6-DOF arrives, rotates by `Pose.orient`).

Both consumers, one resolver, identical numbers

The simulator’s composite/aero code and the visualizer’s mesh walk both obtain every part’s absolute placement from the *same* call to `resolvePlacements`. They no longer each compute an axial station from the stored offset, so they cannot disagree: the number the integrator uses to place a part’s mass is, by construction, the number the renderer uses to draw it. The CM-to-CM versus geometric-center divergence of [Section 1](#) is structurally impossible in the new model.

4.5 Caching: the resolve must not run per ODE step

The composite cache today is gated on mass change. `ensureCompositeCache` (`model/parts/Part.h:288`) rebuilds the cached CM and inertia tensor whenever the structure is dirty *or* the composite mass has moved since the last build, keyed on `builtAtCompositeMass` (`model/parts/Part.h:304`). That gate is correct for mass: while a motor burns, the composite mass differs every integration step, so the mass-delta gate fires every step — exactly when the CM and inertia genuinely need re-weighting.

But the *geometry* does not change during a burn. The station landmarks, the radii, the seat gaps, and therefore every resolved `Pose` are rigid functions of the parts’ lengths and seat intents; they are invariant under mass loss. Re-running the resolver — a full DFS, with the quaternion compositions and the overlap sweep — on every ODE step would be pure waste, and during the heavy full-ladder flight sweeps it would dominate the cost of an otherwise cheap arithmetic update.

The design therefore splits placement into its own cache, gated on *structural* change rather than mass change. The present tree has a single dirty flag, `needsRecomputing` (`model/parts/Part.h:322`), propagated up the ownership chain by `markAsNeedsRecomputing` (`model/parts/Part.h:293`); the refactor adds a *separate* `placementDirty` flag and a `resolvedCache` member.

Two gates, two cadences

`resolvedCache` (a `std::vector<Placed>`) is rebuilt by `ensurePlacementCache` *only* when `placementDirty` is set — by `addChildPart`, `removeChildById`, or a geometry/length edit — and *never* by mass change. `computeCompositeAt(t)` reads the mass-independent `resolvedCache` and re-weights only by `getMass(t)`. A burning motor thus re-runs the cheap mass/CM/parallel-axis arithmetic over *fixed geometry*, while the resolver and the overlap sweep ([Section 6](#)) run once per structural change. The mass-delta gate fires every step; the placement gate fires only when the rocket is actually re-built.

This separation is the operational counterpart of the “derive, don’t store” rule ([Section 2](#)): the resolved poses are *derived* (never serialized, never authored), but they are also *memoized* against the correct invalidation key, so deriving them is not paid for on a cadence the physics does not warrant.

5 Authoring API & the zero-config default

The data model of [Section 3](#) and the resolver of [Section 4](#) are only as good as the surface authors actually touch. This section specifies that surface: the single primary call that attaches a child, the defaults that make the overwhelmingly common case cost nothing, and the optional verbs that name

an intent without ever fixing a coordinate. The governing principle is the third non-negotiable rule of [Section 2](#) — *the common case costs zero authored numbers* — and the API is shaped backwards from it.

5.1 The primary call and the transitional shim

Today the base `Part` exposes exactly one attachment entry point, at `model/parts/Part.h:239`:

```
virtual void addChildPart(std::shared_ptr<Part> child, Vector3 position);
```

whose `Vector3` is the child's center of mass measured relative to the parent's center of mass — the CM-to-CM coupling this design exists to remove ([Section 3](#)). The refactor replaces that signature with a link-valued one and keeps the old one alive, deprecated, only long enough for the phased migration of [Section 8](#) to complete.

```
// Primary API -- replaces the legacy addChildPart(child, Vector3).
// A default-constructed StationLink means "abut aft" (see below), so the
// trivial stack call carries no authored geometry at all.
virtual void addChildPart(std::shared_ptr<Part> child, StationLink link = {});

// Transitional shim, retained only through the migration window so existing
// tuple call sites and every legacy *.qrd fixture still compile. It synthesizes
// a StationLink reproducing the legacy CM-to-CM station and logs a deprecation
// note; it is removed once the corpus is cut over (see migration plan).
[[deprecated]] void addChildPart(std::shared_ptr<Part> child, Vector3 position);
```

Listing 11. The link-valued primary `addChildPart` and the `[[deprecated]]` `Vector3` shim that bridges the migration.

Two properties of [Listing 11](#) are deliberate. First, the new overload is a strict superset of the old one: a default-constructed `StationLink` encodes the most common relationship, so the migration is additive rather than a forced rewrite of every call site at once. Second, the shim is marked `[[deprecated]]` rather than deleted. A hard deletion would break the build the instant the type changes, before any fixture or test could be ported; the attribute instead lets the tree stay green while emitting a compile-time warning at each unported call site, turning the migration into a punch list the compiler maintains. The shim's body is not a behavioral synonym for the new call — it reconstructs the equivalent `StationLink` from the legacy CM-to-CM offset using the explicit cone frame adapter, the sign-correct derivation of which is the subject of [Section 8](#). Once the fixture corpus is re-saved in the new schema, the shim (and with it the last consumer of the CM-to-CM convention) is retired.

5.2 The zero-config default

A `StationLink` is four numbers and a seat kind ([Section 3](#)); the value of the design hinges on what those fields default to. A default-constructed `StationLink{}` carries `parentStation01 = 0` (the parent's aft plane), `childStation01 = 1` (the child's fore plane), `gap = 0`, and `seat = SeatKind::Abut`. Read in the settled $+z$ = forward convention, that means *the child's fore plane meets the parent's aft plane*: a newly attached child stacks $+z$ = aft of its parent.

The default is the build order

A bare `parent->addChildPart(child)` abuts the child's fore plane to the parent's aft plane. This is exactly the natural nose $\rightarrow$ body $\rightarrow \dots$ assembly order: each part is attached behind the one before it, and the resulting stack grows aft from the nose tip without the author writing a single coordinate.

This is correct precisely because the convention was settled once. With $+z$ = forward fixed and stations expressed as explicit fractions of live length, `parentStation01 = 0` is *always* the aft plane, for every part type, with no per-type special case and no ambiguity about which end “0” denotes. The default is therefore not a heuristic to be remembered but a direct reading of the data model. Contrast this with the CM-to-CM scheme it replaces, where the “natural” offset for a nose-to-body abutment was a hand-computed function of both parts' center-of-mass positions — a number that an author could only get right by reproducing the resolver's arithmetic in their head.

Key correctness detail

The default link is the only one of QtRocket's three measured relationships (Section 3) that needs no parameters, and it is the one that occurs most often — every plain axial stack edge. Making it the zero-argument case means the API charges authored numbers only where a real geometric choice is being made (an inset fin station, an insertion depth), never for the trivial mating that dominates a typical design.

Listing 12 renders the three relationships of the `xl75_multi` fixture as authoring calls. The first is the zero-config default; the other two name a genuine geometric intent and pay only for the numbers that intent actually contains.

```
// (1) Body fore plane abuts nose aft plane -- ZERO authored numbers.
nose->addChildPart(body);

// (2) Fin root seats on the body's outer wall near the aft end.
body->addChildPart(fins, {parentStation01 = 0.06, // wall station near body aft
                          childStation01 = 0.0,  // fin root aft
                          seat = SeatKind::OnSurface});

// (3) Coupler nests into the body's fore bore (insertion depth 40 mm).
body->addChildPart(coupler, {parentStation01 = 1.0, // body fore plane (bore mouth)
                             childStation01 = 1.0, // coupler fore plane
                             gap = 0.04,           // insertion depth
                             seat = SeatKind::NestInBore});
```

Listing 12. The three `xl75_multi` relationships as `addChildPart` calls: the defaulted abut, an **OnSurface** fin seat, and a **NestInBore** coupler.

Designated initializers carry their own documentation: each call reads as the sentence the author means, and an omitted field falls back to its zero-config value, so the **OnSurface** fin call need not restate `gap = 0` and the abut call states nothing at all. The fully worked numeric resolution of these three edges — where each plane lands in the root frame, why the body extends aft into $-z$, and how the coupler's forward projection is caught — is given in Section 10; here the point is only that the authored input is minimal and intent-shaped.

5.3 Snap-operator verbs

For authors who prefer a named operation to a brace-initialized aggregate, the design adds a small set of free *snap-operator* verbs. [Listing 13](#) lists them. Their defining property is in the return type: each verb *produces* a `StationLink` and returns it by value. None of them stores a pose, mutates a part, or fixes an absolute coordinate.

```
// Snap-operator verbs: each RETURNS a StationLink describing an intent.
// They never resolve a pose and never store absolute coordinates.
StationLink abut      (double gap = 0.0);      // {0, 1, gap, Abut}  child fore -> parent aft
StationLink nestInBore (double depth);         // {1, 1, depth, NestInBore}
StationLink seatOnWall (double parentStation01); // {parentStation01, 0, 0, OnSurface}

// Usage is identical in effect to the brace form of the worked examples:
body->addChildPart(coupler, nestInBore(0.04));
```

Listing 13. The snap-operator verbs. Each is a pure factory for a `StationLink`; the resolver alone turns intent into geometry.

Verbs name intent; they do not resolve it

A snap verb is a pure factory for a `StationLink` value. It performs no geometry: it neither reads a part’s length nor computes a station’s *z*, and it returns the same kind of stored *intent* that a hand-written brace initializer would. All resolution from intent to absolute placement happens in the one resolver of [Section 4](#), never in the authoring layer.

This separation is what keeps the verbs safe. Because `abut`, `nestInBore`, and `seatOnWall` return data and touch no part, calling one before its operands are sized, or reusing its result across designs, cannot bake in a stale coordinate — the failure mode that storing a resolved transform would reintroduce ([Section 2](#)). They are sugar over the same closed three-relationship vocabulary the brace form exposes: `abut` maps to the zero-config default’s `{0, 1, gap, Abut}`, `nestInBore` to a fore-to-fore `{1, 1, depth, NestInBore}` insertion, and `seatOnWall` to a wall station with the child’s aft plane on the surface. They add no expressive power over [Listing 12](#); they exist solely so that an author may write the relationship as a verb when that reads more clearly than an aggregate, while the type system guarantees that either spelling produces the identical, pose-free `StationLink`.

6 Overlap & constraint diagnostics

Placement that derives center of mass from physical intent ([Section 4](#)) removes one class of error — the simulator and the visualizer can no longer disagree about where a part sits, because they read the same resolved `Pose`. It does not, by itself, remove the other class: a relationship can be geometrically *consistent* yet physically *impossible*. A coupler can be told to nest forty millimetres into a bore that is only thirty millimetres deep; a tube can be seated against a rim of the wrong radius; a part nested near a tube’s fore end can project past that tube into the tapering nose beyond it. The resolver will happily place all of these — the arithmetic is well defined — and the result will be a self-intersecting solid that is meaningless as a rocket. The diagnostics layer is the component that refuses such designs, and refuses them with a *located* reason rather than a silent miscomputation.

The design splits this work into three escalating layers, ordered cheapest-first, and feeds a single verdict structure that gates both downstream consumers. The verdict is computed once per structural

resolve and cached alongside the placement cache (Section 4), so the gate that protects the per-step ODE loop costs nothing per step. Figure 7 traces the control flow from a structural edit through the three layers to the cached `SolveResult` and the two consumer gates.

6.1 The verdict structures

The two POD types live in the new `model/parts/Placement.h` alongside the resolver (Section 3). An `OverlapDiagnostic` is a single *located* violation: who intrudes, into what, where, and by how much. A `SolveResult` is the aggregate verdict — a boolean the consumers can branch on, plus the full diagnostic list for surfacing to the author.

```
struct OverlapDiagnostic {
    Part::Id    offender;
    Part::Id    host;           // the part it intrudes into (may be a non-tree
                                // neighbour, e.g. the nose)
    double      zWorld;         // located station of worst violation (root frame, +z fwd)
    double      penetration;    // metres the offender radius exceeds host capacity
    std::string message;
};

struct SolveResult {
    bool ok{true};
    std::vector<OverlapDiagnostic> diagnostics;
};
```

Listing 14. The diagnostics verdict types. `offender` and `host` are `model/parts/Part.h:56 Part::Id` values (`std::uint64_t`), the stable per-instance identifiers retrieved via `getId()` (`model/parts/Part.h:191`); every instance, including a `clone()` copy, carries a distinct id (a copy is re-minted to a fresh one, `model/parts/Part.h:188`) that never changes once assigned, so attribution is unambiguous even when two parts share a human-facing name (which need not be unique, `model/parts/Part.h:193`).

The choice to identify offender and host by `Part::Id` rather than by name or by pointer is deliberate. Names are explicitly not required to be unique (`model/parts/Part.h:193`), so a name-keyed diagnostic could point at the wrong part; raw pointers would dangle if the tree were edited between the resolve and the report. The `Part::Id` is assigned once at construction, re-minted on copy, and never changed, which makes it the only safe handle to carry in a cached verdict.

6.2 Layer 1 — radial seam check, on every resolve

The first layer runs as each child is placed, at the moment its `StationLink` is resolved against its parent. It is a single radius comparison between the two mated stations, dispatched on the `SeatKind` that the link declares, because the seat kind is precisely what fixes *which* two radii must agree and *how*. Using the worked corpus fixture `tests/data/designs/xl75_multi.qrd:1` for concrete numbers:

- **Abut:** the two outer radii must match — $|p.rOuter - c.rOuter| \leq tol$. The nose base rim (0.0395 m) equals the body fore rim (0.0395 m), so the seam is clean. **OK**
- **OnSurface:** the child's outer radius must equal the parent's outer radius at the seat station — the fin's `bodyRadius` (0.0395 m) against the tube OD (0.0395 m). **OK**
- **NestInBore:** the child's outer radius must fit inside the parent's bore — $c.rOuter \leq p.rInner + tol$. The coupler OD (0.0376 m) is no larger than the body ID (0.0376 m), so it fits. **OK**

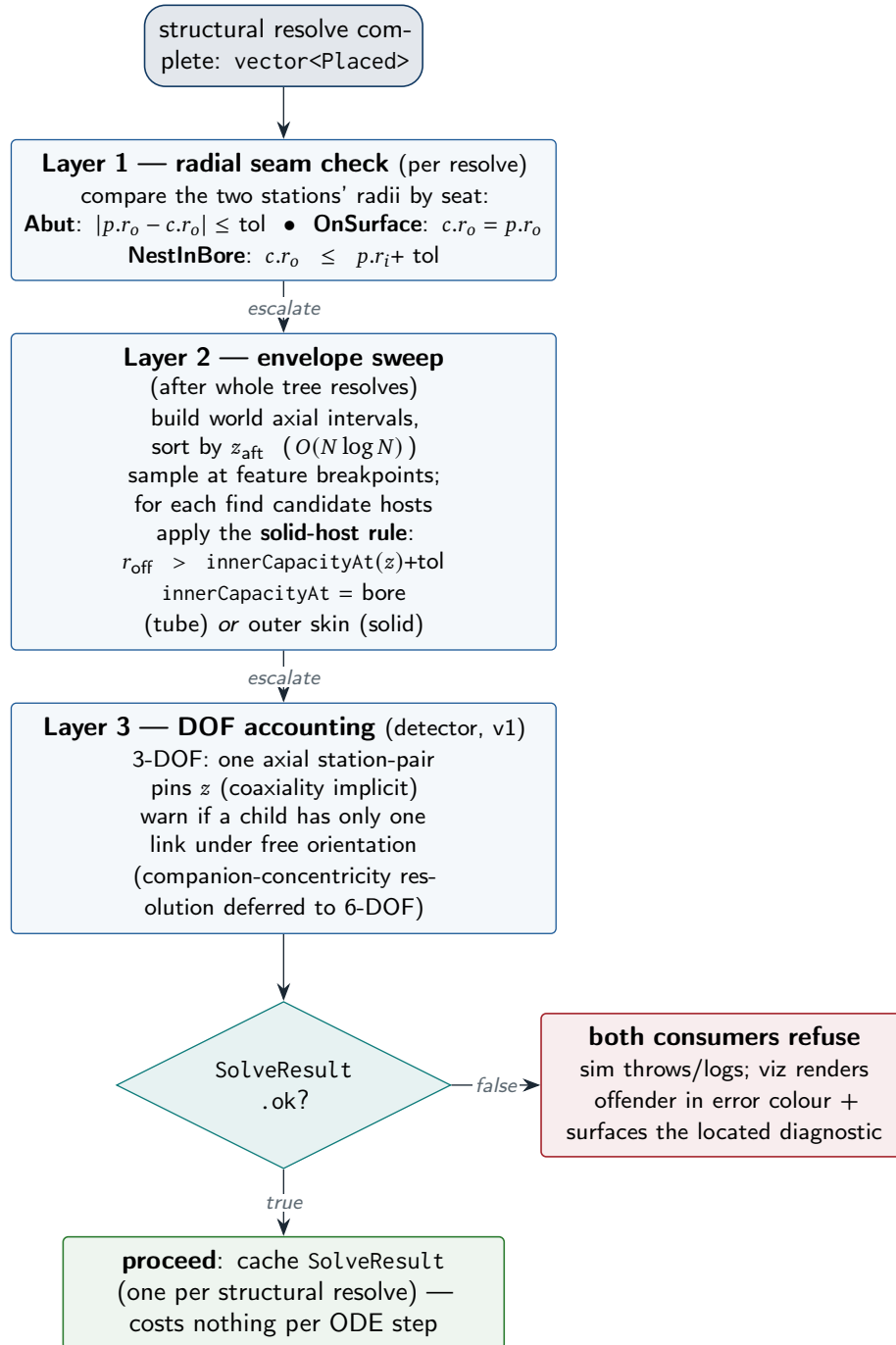


Figure 7. The diagnostics pipeline: one structural resolve runs the three escalating layers and caches a single `SolveResult`; both the simulator and the visualizer consult that cached verdict before they will solve or render.

Layer 1 is local and $O(1)$ per attachment. It catches the common authoring mistakes — mating mismatched rims, nesting an over-wide coupler — at the seam, before any global query runs, and at essentially no cost. What it cannot see is interference between parts that are not directly mated: a part placed at one seam can still collide with a third part somewhere else along the axis. That is Layer 2’s job.

6.3 Layer 2 — envelope sweep, after the whole tree resolves

The spatial query

Layer 2 is a concrete one-dimensional spatial query over the fully resolved tree. Once `resolvePlacements` has produced its `std::vector<Placed>` in deterministic DFS order, the sweep builds a world-frame axial interval $[z_{\text{aft}}, z_{\text{fore}}]$ for every part with a non-trivial envelope and asks, at every feature breakpoint, whether any part’s outer radius exceeds the radial capacity available to it at that station.

The query proceeds in three steps.

1. Build and sort intervals. From each `Placed` pose, the part’s local span $[0, \text{axialLength}]$ is mapped to a world interval $[z_{\text{aft}}, z_{\text{fore}}]$. The intervals are sorted by z_{aft} in $O(N \log N)$. Sorting once lets every subsequent host lookup be a sweep over an active set rather than a quadratic scan, which matters because the sweep is the most expensive layer and must remain comfortably sub-quadratic for realistic designs.

2. Sample at feature breakpoints and find hosts. For each offender, its world span is sampled not at uniform steps but at *feature breakpoints*: its own two endpoints, plus every other interval endpoint that falls within its span. This is exact rather than approximate because each part’s radius profile $r(z)$ is closed-form and monotone — a cone tapers linearly, a tube is constant, a sphere bulges — so a thin protrusion cannot slip between samples and go unnoticed. At each sample station z_{world} , candidate hosts are found by advancing through the sorted interval list (the active set of intervals covering that station). The interference test at each candidate is

$$\text{offender.radiusOuterAt}(z_{\text{local}}) > \text{host.innerCapacityAt}(z_{\text{local}}) + \text{tol},$$

applying the *solid-host rule* established in the data model (Section 3). That rule is what makes a single predicate correct for both bored and solid hosts: `innerCapacityAt` returns the bore radius (`radiusInnerAt`) for a part with a wall, and the outer-skin radius (`radiusOuterAt`) for a solid. A naive “offender OD versus host inner radius” test would read 0 for every solid part — whose `radiusInnerAt` is 0 — and flag everything; the solid-host rule instead compares the offender against the solid’s outer skin, which is exactly the poke-through test one wants.

3. Deterministic tie-break. When more than one interval covers a sample station, the host is chosen as the covering part with the *smallest* `Part::Id`, excluding the offender itself and the offender’s own ancestor chain through the seat it legitimately occupies. Because ids are assigned at construction and re-minted deterministically in DFS load order (Section 3), this tie-break is reproducible across runs and across reloads of the same design — a property the diagnostics must have if a reported violation is to be debuggable.

Why the sweep walks into a non-tree neighbour

The host of a violation need not be the offender's parent, or any ancestor at all. Consider the worked coupler, nested at the body's fore end with a 0.04 m insertion depth so that its fore plane projects forward *past* the body fore rim. Over the world span $z \in [0, +0.04]$ the offender lies forward of the body and inside the nose region. The sorted-interval query at those samples finds the host to be the *nose cone* — a non-tree neighbour the coupler is never linked to. The solid-host rule evaluates the nose's tapering skin, `nose.radiusOuterAt`, at each station; the coupler OD (0.0376 m) exceeds the nose skin (0.0342 m at $z = 0.04$), and the forward poke-through is flagged with a located z_{world} . The global sweep sees the collision precisely because it does not restrict itself to the ownership tree.

FinSet, handled honestly. A `FinSet` is not axisymmetric, so it has no scalar outer radius the sweep could meaningfully consume. The design treats it accordingly rather than pretending otherwise. For *axial* occupancy the fin set contributes only its body-disc radius — its `getBodyRadius()` (`model/parts/FinSet.h:67`), the same disc it reports as its reference area (`model/parts/FinSet.h:72`) — so it hosts nothing inside it and is not falsely modelled as a `bodyRadius + span` disc that would collide with every adjacent body tube. The radial fin-to-fin and fin-to-tube interference — the `span` extent occupying azimuthal sectors — is *explicitly scoped out of v1* and documented as unchecked.

A full `bodyRadius + span` disc for fins

Reporting a fin set's outer radius as `bodyRadius+span` — its true tip extent, `getMaxRadius()` (`model/parts/FinSet.h:73`) — would let the existing sweep “check” fins for free, but dishonestly. The tip extent occupies only a few azimuthal sectors, not a full disc; modelled as a solid disc it would produce a false positive against every body tube it passes. Honest sectorised interference needs a fin clocking/azimuth model that QtRocket does not yet have, so v1 declines to fake it. Layer 1 still verifies fin-root radial compatibility, and the fin set's axial placement is exact. The geometry needed for the future sectorised check — `getSpan()` (`model/parts/FinSet.h:64`), `getSweep()` (`model/parts/FinSet.h:65`), `getTipChord()` (`model/parts/FinSet.h:63`) — remains available for when that model lands.

6.4 Layer 3 — DOF accounting

The third layer is forward-looking. In the 3-DOF regime that QtRocket integrates today, a single axial `StationLink` fully pins a child's position, because coaxiality is implicit — every part shares the body-frame orientation and sits on the longitudinal axis ([Section 9](#)). One axial constraint is therefore sufficient, and the layer simply records that invariant.

Once 6-DOF is enabled, that is no longer true. A single `StationLink` cannot express “concentric to a third part”: under free orientation a child constrained by one link has an unpinned rotational degree of freedom. The full resolution — a companion concentricity constraint — requires the deferred joint graph ([Section 11](#)) and is out of scope for v1. What v1 *does* provide is the detector: Layer 3 emits a warning when a child has only one link under free orientation, so that the missing constraint is surfaced rather than silently producing an under-determined placement. The detector is cheap and the warning is honest about what it can and cannot fix.

6.5 The gate

SolveResult.ok == false stops both consumers

When the diagnostics produce `SolveResult.ok == false`, *both* downstream consumers refuse the solve. `computeCompositeAt` (`model/parts/Part.cpp:169`), the routine the simulator drives every ODE step through `ensureCompositeCache` (`model/parts/Part.h:288`), throws and logs rather than returning an inertia computed over a self-intersecting solid. `RocketMesh::walk` (`visualizer/RocketMesh.cpp`, the DFS at line 427) renders the offender in an error colour and surfaces the diagnostic message instead of drawing the impossible geometry. The coupler-through-nose case is thus a hard, located failure in *both* views — never the silent render of the legacy `tests/data/designs/xl75_multi.qrd:1` fixture, where each consumer drew the interpenetration without complaint.

The gate is correct only because it is also cheap. If the diagnostics ran per ODE step they would dominate the integration cost; if they ran lazily inside `computeCompositeAt` they would re-fire on every mass-delta rebuild during a burn (the mass-delta gate `ensureCompositeCache` fires every step while the motor mass changes, `model/parts/Part.h:288`). Neither happens. The `SolveResult` is produced exactly once per structural resolve — when a part is added or removed, or a geometry edit changes the tree — and cached alongside the placement cache described in [Section 4](#). Each consumer reads the cached boolean: the simulator branches on it before re-weighting the cached geometry by `getMass(t)`, and the visualizer branches on it before walking the resolved poses. The verdict is computed where the geometry changes, not where it is consumed, so guarding the inner loop is a single boolean test.

7 Serialization to .qrd

The on-disk `.qrd` format must carry the same shift the rest of this design makes everywhere else: it stops recording resolved geometry and starts recording *intent*. Today a child’s spatial relationship to its parent is persisted as an opaque `<offset x y z/>` triple — the serialized form of the CM-to-CM `Vector3` stored in `childParts`. That triple inherits every pathology discussed in [Section 3](#): it is a resolved coordinate, it conflates the cone’s internal CM frame with the assembly frame, and it bakes a particular set of part lengths into the file so that a later geometry edit silently invalidates it. Replacing the storage type ([Section 3](#)) without replacing its serialized projection would merely move the bug onto disk. This section defines the replacement element, the reader/writer changes, and why the format version need not gate the loader.

7.1 The `<link>` element, kept inline

The `<offset>` child element is replaced, per child, by an inline `<link>` element that serializes the -bearing `StationLink` rather than a resolved vector. The element is attached to the child `<part>` it governs, exactly where `<offset>` sat, and it carries the four authored quantities of the link: the parent station, the child station, the gap, and the seat kind.

The link is stored inline with its child, not in a sibling block

A tempting alternative — collecting all parent→child relationships into a single sibling `<joints>` block keyed by part name — is a hazard, because part *names need not be unique*: `getName()` is explicitly documented as non-identifying (`model/parts/Part.h:193`; identity is the per-instance `getId()`, which is not serialized). A name-keyed joints block would therefore be ambiguous the

moment a design contains two parts with the same name, which is a permitted and common state. Anchoring each `<link>` to its owning child element sidesteps name resolution entirely: the ownership tree in the XML is the keying.

Listing 15 shows the target 0.2 encoding of the `xl75_multi` design, with each of its three non-root attachments expressed as a `<link>`.

```
<QtRocketDesign version="0.2">          <!-- minor bump; loader still accepts 0.1 -->
  <part type="NoseCone" name="MultiNose">
    <params baseRadius="0.0395" length="0.30" wallThickness="0"
            density="1700" solid="true"/>
    <!-- root: no <link> -->
    <children>
      <part type="BodyTube" name="MultiBody">
        <params innerRadius="0.0376" outerRadius="0.0395" length="0.90"
                density="1700"/>
        <link seat="Abut" parentStation="0.0" childStation="1.0" gap="0"/>
        <!-- ^ equals the default link; omissible -->
        <children>
          <part type="BodyTube" name="MultiCoupler">
            <params innerRadius="0.036" outerRadius="0.0376" length="0.08"
                    density="1700"/>
            <link seat="NestInBore" parentStation="1.0" childStation="1.0"
                  gap="0.04"/>
          </part>
          <part type="FinSet" name="MultiFins">
            <params rootChord="0.10" tipChord="0.04" span="0.06" sweep="0.04"
                    thickness="0.003" bodyRadius="0.0395" finCount="6"
                    density="1700"/>
            <link seat="OnSurface" parentStation="0.06" childStation="0.0"
                  gap="0"/>
          </part>
        </children>
      </part>
    </children>
  </part>
  <motor commonName="M1350W"/>
  <sim .../>
</QtRocketDesign>
```

Listing 15. The `xl75_multi` design in the target 0.2 format. Each child’s spatial relationship is an intent-bearing `<link>`, never a resolved coordinate. The root carries no link; the body’s link equals the default and could be omitted.

7.2 Writer: `writeOffset` becomes `writeLink`

The current writer emits the resolved offset through `writeOffset` (`model/DesignSerializer.cpp:53`), which puts `x/y/z` attributes from a `Vector3`; `writePart` adds that node as the child’s `offset` sub-element. Under the new model this helper is replaced by `writeLink(const StationLink&)`, which emits the four link attributes — `seat`, `parentStation`, `childStation`, and `gap` — in place of the coordinate triple. The enumerator is mapped to and from its attribute string through a single shared lookup table, so the writer and the reader agree on the spelling (`Abut`, `NestInBore`, `OnSurface`) by

construction rather than by two independently maintained switch statements.

A default-equal link is omissible

Because the zero-config default is a meaningful physical relationship — the child’s fore plane abutting the parent’s aft plane (Section 5) — a link whose fields all equal that default carries no information and may be omitted from the file entirely. The writer can elide it, and the reader (below) reconstructs the same default. This keeps the common nose→body→...stack as terse on disk as it is in the authoring API, and it makes the serialized form proportional to the design’s actual complexity rather than to its part count.

7.3 Reader: buildPart consumes the link

The reader’s recursion lives in `buildPart` (`model/DesignSerializer.cpp:117`). Today it reconstructs a `Vector3` from the child’s `offset.<xmlattr>.x/y/z` (defaulting each to `0.0`) and calls the legacy `addChildPart(child, Vector3)`. Under the new model it instead reads the link attributes — `parentStation`, `childStation`, `seat`, `gap` — defaulting them respectively to `0.0`, `1.0`, **Abut**, and `0`, then constructs a `StationLink{...}` and calls `addChildPart(child, StationLink{...})`. Those defaults are exactly the zero-config link, so a minimal `<part>` carrying *no* `<link>` at all still attaches by abutting its parent — the omissible-link property of the writer and the defaulting behavior of the reader are two halves of the same contract.

The reader inherits the existing fail-closed discipline. A `seat` string that maps to no enumerator is a load error, raised on the same path as today’s `makePart` rejection of an unknown part type, so a malformed or hand-edited file is refused with a clear diagnostic rather than silently coerced to **Abut**.

Resolved coordinates are never serialized

The file stores only authored intent — station fractions, a gap, and a seat kind. The absolute `Pose` of each part is recomputed by the resolver (Section 4) on load. This is the on-disk expression of the “derive, never store” rule: a .qrd file can be edited (lengthen a tube, swap a cone) and reloaded, and the seams move to follow the new geometry because nothing in the file pins them to the old. It is also the natural validation point for migration — recomputing the placement on load is precisely when the overlap sweep (Section 6) gets to run against a freshly read tree.

7.4 Version: writer emits 0.2; no loader gate change

The format version moves from `0.1` to `0.2` to mark the `<offset>→<link>` change. This is a minor bump because the loader’s acceptance rule is already keyed on the *major* component only: the gate (`model/DesignSerializer.cpp:186`) splits the version on the dot and rejects only an unrecognized major, so it admits any minor within major `0`. Both `0.1` (legacy `<offset>`) and `0.2` (`<link>`) therefore pass the same gate with no change to it; the reader distinguishes the two element shapes structurally, by which element is present on the child, not by inspecting the version string.

The current writer still emits 0.1

In the present working tree the writer hard-codes the version string `"0.1"` (`model/DesignSerializer.cpp:156`), and `writeOffset` still emits the coordinate triple. The `0.2/<link>` encoding described above is the *target* of this design, not the current behavior; the only file-format work the loader gate needs is none, precisely because `model/DesignSerializer.cpp:186` already tolerates the minor bump.

8 Migration plan

The refactor described in the preceding sections rewrites the very heart of the composition math—the meaning of the stored child offset, the input to `model/parts/Part.cpp:169`’s two-pass mass/CM/parallel-axis loop, and the axial station threaded through `model/parts/Part.cpp:214`’s aero walk. A change of that reach to a shipping simulator cannot be validated by inspection. The migration is therefore structured around a single load-bearing claim: the new geometry-driven placement model is *physics-invariant* on every existing design. The center of mass, the composite inertia tensor, and every resolved axial station that the legacy CM-to-CM reader produces must be reproduced *bit-for-bit* by the migrated reader. The plan is consequently analytic rather than empirical—no flight re-runs, no tolerance fudging—and it is gated by a test that is written and made to pass *before* any production type changes.

The four phases below are ordered so that the build stays green at every step and the invariance claim is proven, not assumed.

8.1 Phase 0 — the invariance gate, written first

Before a single type changes, a test loads every fixture in `tests/data/designs` (the 24 `.qrd` designs spanning the `micro13` through `xl75` ladders) through the *current* reader and snapshots three quantities per design: `getCompositeCm(0)` (`model/parts/Part.h:134`), `getCompositeI(0)` (`model/parts/Part.h:146`), and the resolved axial station of every part in the existing `+z` = forward CM-to-CM convention.

The discipline that makes this a *gate* rather than a self-fulfilling check is that the snapshot is taken from the legacy reader’s own output, not re-derived from the design’s intended geometry. A re-derivation could silently share whatever bug the legacy path contains; the legacy reader’s literal output cannot. These frozen values become the ground truth the migration is held against.

Invariance is defined by the incumbent, not the spec

The Phase-0 snapshot is whatever the current CM-to-CM code computes today, including quantities the new model would consider mis-placed. The migration’s job is to reproduce the incumbent numerics exactly; *correcting* a design’s geometry (the `xl75_multi` poke-through) is deferred to Phase 3, where it is done deliberately and visibly, not folded silently into the type swap.

8.2 Phase 1 — types and resolver land, both APIs coexist

Phase 1 introduces the new machinery alongside the old without removing anything the build depends on. The new header `model/parts/Placement.h` (Section 3) lands with **Abut/NestInBore/OnSurface**, the `StationLink`, `Station`, and `Pose` value types, and the free-function resolver and overlap sweep (Section 4, Section 6). The geometry virtuals—`stationAt`, `radiusOuterAt/radiusInnerAt`, `axialLength`, `isSolid`, `cmStationLocal`—are added to the base `Part` and overridden on the four concrete part types. The single child-storage slot at `model/parts/Part.h:326` is swapped in place from `std::tuple<std::shared_ptr<Part>, Vector3>` to `std::pair<std::shared_ptr<Part>, StationLink>`, and `model/parts/Part.cpp:169` and `model/parts/Part.cpp:214` are rewritten to consume the resolver’s geometric pose.

Crucially, the legacy authoring surface survives. The current sole overload `model/parts/Part.h:239`, `addChildPart(std::shared_ptr<Part>, Vector3)`, is retained as a `[[deprecated]]` shim so that every existing tuple-style call site and every legacy `.qrd` fixture still compiles and still produces the

legacy placement. The shim is the analytic core of the migration: it must synthesize a `StationLink` whose resolved geometry reproduces the legacy CM-to-CM offset exactly.

8.2.1 The CM-to-CM recovery formula

The legacy `Vector3` is the child's CM expressed relative to the parent's CM. The resolver, by contrast, places a child by its *aft-plane origin* in the parent frame (Section 4). The shim bridges the two by inverting the CM-to-station relationship, working entirely in the $+z$ = forward frame so that no axis is mirrored. Listing 16 gives the recovery.

```
// All quantities in +z = forward. legacyOffset.z is the child CM minus the
// parent CM (the legacy CM-to-CM coupling).
//
// 1. Recover the child's aft-plane origin in the parent frame. *CmStationLocal
//    is cmStationLocal() evaluated in the +z-forward local frame; the cone is
//    the only part whose adapter differs (see the cone frame adapter).
childOriginZ_fwd = parentCmStationLocal - childCmStationLocal + legacyOffset.z;

// 2. Choose a station pair, then back the gap out of the resolved geometry.
//    childStation / parentStation are the picked Station landmarks.
gap_unsigned = childOriginZ_fwd + childStation.z - parentStation.z; // unsigned

// 3. Infer the seat from the PART PAIR -- BEFORE applying any sign:
//    tube fore-rim against tube aft-rim      => Abut
//    small-tube OD inside large-tube ID      => NestInBore
//    fin root on the wall                    => OnSurface
seat = inferSeat(parent, child);

// 4. Apply the seat sign LAST. NestInBore inserts aft (-z); all others stand
//    off forward (+z).
gap = (seat == SeatKind::NestInBore) ? -gap_unsigned : +gap_unsigned;
```

Listing 16. The `[[deprecated]]` shim's analytic recovery of a `StationLink` from a legacy CM-to-CM `Vector3`. The `cmStationLocal` adapter is consumed here (its first production use); the cone supplies the only non-trivial override.

Two details in Listing 16 carry the correctness of the entire migration, and both are easy to get backwards.

$+z$ is forward throughout — no mirror

The legacy `xl75_multi` coupler stores $z = +0.49$ relative to the body (`tests/data/designs/xl75_multi.qrd:19`). Because the convention is $+z$ = forward on both sides of the recovery, that $+0.49$ is read directly as *forward*—toward the nose tip—with no sign flip and no whole-rocket inversion. The migration is a pure re-expression of the same axis, which is precisely why $+z$ = forward was chosen over $+z$ = aft (Section 2): $+z$ = aft would have forced a sign-negating rewrite of every fixture here. The cone is the sole part whose *internal* CM frame (`coneCmOffset`, `model/parts/ConicalNoseCone.cpp:49`) opposes the forward convention, and that single discrepancy is absorbed by the `cmStationLocal` adapter (Section 3), never by mirroring the link.

Seat is inferred before the gap sign

The seat must be determined from the part pair *before* the gap sign is applied, because **NestInBore** is the one seat that flips the sign (step 4 of [Listing 16](#)). Computing an unsigned gap first, then choosing the sign from a seat inferred independently of that gap, is what makes the “reproduces the exact station” invariant hold for *every* seat—not only the sign-preserving **Abut** and **OnSurface** cases. Inferring the seat from the already-signed gap would be circular and would mis-place every nested coupler.

By the end of Phase 1 the build is green, the new and legacy APIs coexist, and the shim reproduces legacy behavior analytically.

8.3 Phase 2 — run the gate against the migrated reader

Phase 2 loads every fixture through the now 0.2-capable reader—legacy `<offset>` elements flow through the shim of [Listing 16](#) into `StationLinks`—and asserts *bit-identical* `getCompositeCm(0)`, `getCompositeI(0)`, and resolved axial stations against the Phase-0 snapshots. Equality is exact, not within a tolerance: the migration is a re-expression of the same arithmetic over the same geometry, so any deviation is a defect, and a tolerance would merely hide it.

A dedicated sub-test pins the nose CM to a hand-computed value as the specific guard that the cone frame adapter is correct. For the `xl75_multi` nose—a solid cone of length $L = 0.30$ (`tests/data/designs/xl75_multi.qrd:5`)—the centroid sits at $L/4$ forward of the base, i.e. `cmStationLocal = 0.075`. This is the forward-frame image of `coneCmOffset`’s internal $z = L/2 - \bar{h}$ with $\bar{h} = L/4$ (`model/parts/ConicalNoseCone.cpp:49`); the adapter maps it to $L/2 - (L/2 - \bar{h}) = \bar{h} = L/4$ ([Section 3](#)). Pinning this one scalar to a value computed by hand—independently of both the legacy and migrated code paths—is the test that catches a sign or frame error in the single most correctness-critical conversion in the design.

Why the cone gets its own hand-checked sub-test

A symmetric part has its CM at mid-length, so `cmStationLocal` is $L/2$ and any frame error cancels by symmetry. The cone is the only current part whose CM is off-center, so it is the only part for which a forward-vs-internal frame mistake would survive the bit-identical aggregate check yet still be wrong. The hand-computed `0.075` isolates exactly that risk.

8.4 Phase 3 — cut over the fixtures and retire the shim

With invariance proven, Phase 3 re-saves the corpus to version 0.2 ([Section 7](#)), replacing each `<offset>` with an explicit `<link>`. Here the migration deliberately stops preserving the past. The `xl75_multi` coupler—legacy-placed forward into and through the nose by its `+0.49` CM-to-CM offset—migrates to a **NestInBore** link, and at that point its Layer-2 envelope sweep *fires*: the coupler outer radius exceeds the tapering nose skin forward of the body fore rim, and the resolver returns `solve.ok == false` with a located `OverlapDiagnostic` ([Section 6](#), [Section 10](#)).

The migration reports the pre-existing bug instead of preserving it

The legacy file drew an interpenetrating coupler silently because the simulator and the visualizer each consumed the bad offset in their own divergent way ([Section 1](#)). Once the relationship is expressed as “nest into the body’s fore bore,” **NestInBore** constrains the insertion direction and

the envelope sweep makes the forward poke-through a hard, located **FAIL**. The correct response is not to suppress the diagnostic but to fix the fixture—reduce the insertion depth or shorten the coupler—until it resolves **OK**.

After each fixture resolves clean, the invariance gate is re-run against the fresh 0.2 files to confirm the corrected designs are stable under reload. Only then is the `[[deprecated]]` overload at `model/parts/Part.h:239` removed, collapsing the authoring surface onto the single `StationLink` API (Section 5) and completing the cutover. The accessor side—`getChildParts()` returning a `pair` rather than a `tuple`—is migrated explicitly at its enumerated call sites (Section 12); the shim covers only the setter.

Figure 8 summarizes the phased flow and the gate that ties it together.

9 The 6-DOF path

The placement model defined in Section 3 and resolved in Section 4 is, by construction, a three-degree-of-freedom (3-DOF) system: every part is seated coaxially, child origins differ from their parents only by an axial translation, and no relative orientation is modeled. That restriction is not accidental — it matches what the current integrator can use (Section 1) — but it is a deliberately *soft* restriction. The data model was shaped so that the eventual move to six degrees of freedom (6-DOF) costs *no schema change, no signature change, and no second resolver*. This section documents exactly which seams were widened in advance, why each widening is free today, and what the one structural code change will be when the rotational degrees of freedom are switched on. Nothing here is implemented or exercised in the 3-DOF cut; it is the forward-compatibility argument for the choices already made.

The motivating constraint is stated in the source itself. The composite mathematics today rests on an explicit frame assumption: “all parts share the same body-frame orientation, so child position offsets are pure translations and tensors combine by addition (no rotation) ... relative part rotation is not modeled” (`model/parts/Part.h:43–45`). The value of the 6-DOF shaping is that it converts this from a structural assumption baked across the tree into a single localized limitation — one identity-valued quaternion and one composition line — that can be lifted in place.

9.1 Orientation already lives in the value types

Two carriers for orientation are present in the data model from day one, both holding the identity quaternion in the 3-DOF cut so that behavior is bit-identical to a pure-translation tree.

The first is `Pose`, the resolver’s output type (Section 4). It pairs the part-local origin with an orientation, and its `compose` operator already rotates the child translation by the parent orientation rather than merely adding it:

Because `orient` is `Quaternion::Identity()` everywhere in 3-DOF, the rotation `orient * childInThis.origin` is the identity map and `compose` degenerates to vector addition. The operator is written correctly for the rotated case now, so no edit to the resolver’s spine is required later; the rotations simply stop being identities.

The second carrier is `StationLink::childRot`, the per-seam orientation defined in the link struct (Section 3). It is a *value* in the already-existing struct, not a new field introduced by the 6-DOF work — the storage layout does not reshape. It defaults to `Quaternion::Identity()` and is consumed by the resolver as the child’s local orientation relative to its parent. This is where canted fins and side boosters — parts that genuinely seat at a fixed relative angle — are expressed: a non-identity

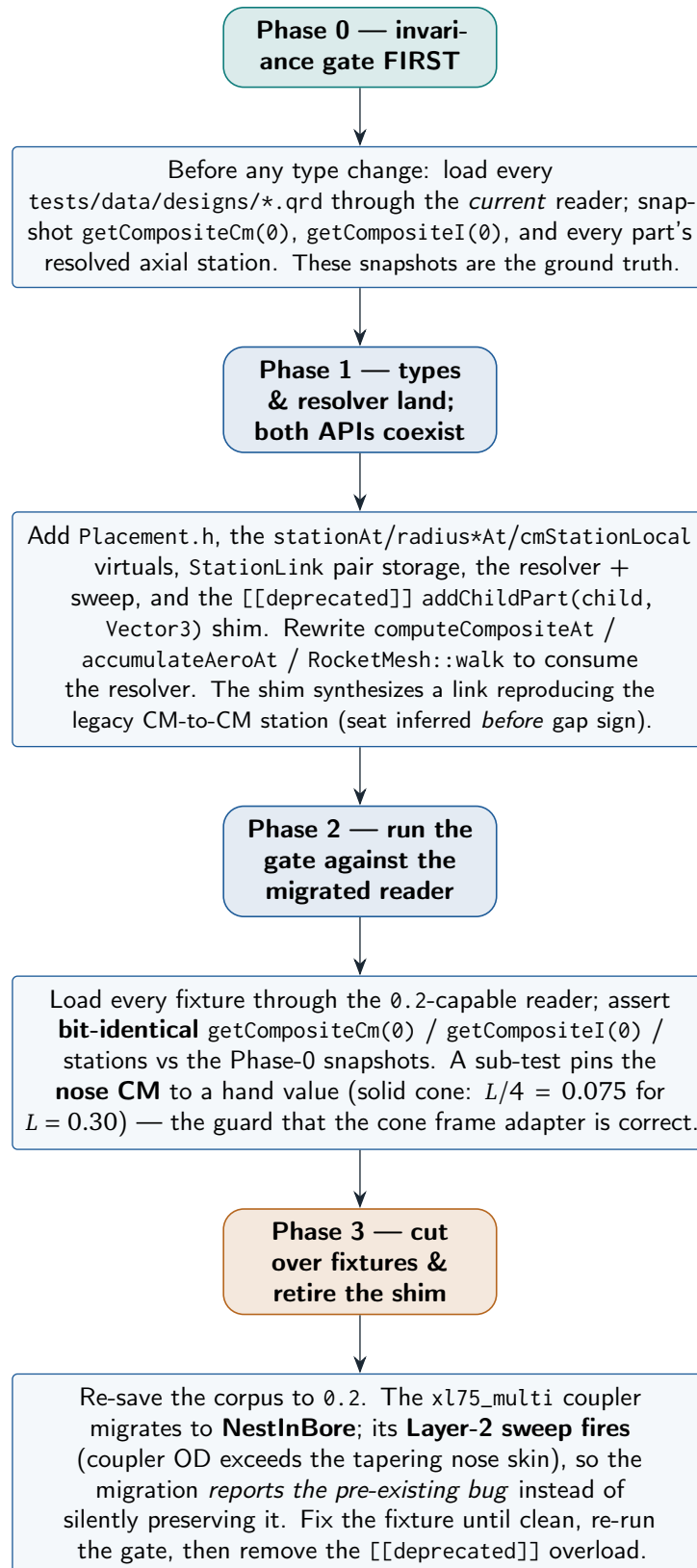


Figure 8. The four-phase migration: the invariance gate is built first (Phase 0), the new types and the analytic shim land with both APIs coexisting (Phase 1), the gate is run against the migrated reader and the cone CM is pinned by hand (Phase 2), and the fixtures are cut over—surfacing and fixing the pre-existing poke-through—before the deprecated shim is retired (Phase 3).

```

struct Pose {
    Vector3    origin {Vector3::Zero()};           // aft-plane origin, in ROOT frame
    Quaternion orient {Quaternion::Identity()};    // (x,y,z,w); identity in 3-DOF

    Pose compose(const Pose& childInThis) const {
        return Pose{ origin + orient * childInThis.origin,
                    (orient * childInThis.orient).normalized() };
    }
};

```

Listing 17. `Pose::compose` rotates the child translation by the parent orientation. With every `orient` equal to identity, `orient * childInThis.origin` is the input unchanged, so the composed pose is a pure translation — bit-identical to the current tree.

`childRot` on the seam, resolved through `compose` into a world orientation, with no other part of the pipeline aware that anything changed.

Zero schema cost is the point

Carrying `orient` on `Pose` and `childRot` on `StationLink` from the first commit means the on-disk `.qrd` schema (Section 7), the storage type for `childParts`, and the resolver signature `resolvePlacements(const Part&, const Pose&)` are all already 6-DOF-shaped. Enabling rotation adds behavior, not types. A design that bolted orientation on later would have to migrate every stored design and re-break every consumer of `getChildParts()` a second time — precisely the churn this whitepaper exists to avoid.

9.2 The one new composition line: rotating the inertia tensor

The single structural change that distinguishes 6-DOF from 3-DOF lives in the composite derivation, not the resolver. The two-pass mass-weighted CM and parallel-axis routine in `computeCompositeAt` (`model/parts/Part.cpp:169–204`) is preserved verbatim by the placement refactor (Section 4); only its *input* changes from stored CM-to-CM offsets to resolver-derived geometric stations. The 6-DOF step adds exactly one operation to that preserved routine: before a child’s inertia tensor is shifted to the composite CM by the parallel-axis theorem (`parallelAxisTerm`, `model/parts/Part.cpp:17–23`), it must first be rotated out of the child’s body frame into the root frame.

```

// In computeCompositeAt's pass 2, per child, with R = childPose.orient:
const Matrix3 R      = childPose.orient.toRotationMatrix();
const Matrix3 Irot = R * childI * R.transpose(); // I' = R I R^T (NEW for 6-DOF)
const Vector3 d     = childCmInRoot - compositeCm; // child CM -> composite CM
I += Irot + childMass * parallelAxisTerm(d);      // parallel-axis shift, unchanged

```

Listing 18. The lone 6-DOF addition to the composite math: the similarity transform $I' = R I R^T$ re-expresses the child’s centroidal tensor in the root frame before the parallel-axis shift. In 3-DOF $R = I_3$, so `Irot == childI` and the line is an identity.

The transform $I' = R I R^T$ is the standard congruence that re-expresses a centroidal inertia tensor under a change of body orientation, where R is the rotation matrix of `childPose.orient`. It must precede the parallel-axis shift because the displacement tensor `parallelAxisTerm` operates on the offset `d` in the root frame; the child tensor must be in that same frame before the two are summed.

In the 3-DOF cut R is the identity, so `Irot` equals `childI` bit-for-bit and the routine reduces to exactly the addition it performs today. This is why `model/parts/Part.h:43–45` can be cited as the *current* limit rather than a permanent one: that “no rotation” assumption is now localized to this single line, and lifting it is a one-line change to a routine whose structure is otherwise untouched.

9.3 Off-axis radial seating arrives with rotation, not before

Section 4 deliberately scopes the radial coordinate r to the seam and overlap *check* only: the pose solve sets $x = y = 0$ and every part is seated coaxially. Off-axis radial placement — a single rail button, an asymmetric pod, a clustered side booster offset from the centerline — is not a 3-DOF feature held back arbitrarily. It is physically meaningful only once two ingredients are present together: `childRot` (so the off-axis part is correctly *oriented* at its radial station, not merely translated to $x = r$ while still pointing along the parent axis) and a per-part body-frame center-of-mass offset.

The second ingredient is the latent purpose of `getCenterMassOffset()` (`model/parts/Part.h:96`). That accessor returns the part’s `cm` member, which is documented as not currently consumed by the composition math but kept as “the natural home for that offset once asymmetric parts or 6-DOF force application ... need it” (`model/parts/Part.h:310–316`). In the 3-DOF cut its sole consumer is the cone CM-frame adapter `cmStationLocal` (Section 3). Under 6-DOF it gains its second documented use: the radial components of the body-frame CM offset, rotated by `childPose.orient` alongside the axial `cmStationLocal` term, place a genuinely off-axis part’s mass correctly in the root frame.

Do not store a half-correct off-axis placement

Seating a part at $x = r$ in 3-DOF — with no orientation and no body-frame CM offset — would record a placement that is correct in translation but wrong in attitude and inertia. That is exactly the staleness-and-divergence error class this design exists to eliminate (Section 2). Off-axis seating therefore lights up *here*, with `childRot` and the body-frame CM offset integrated together, and not one step sooner. Until then r drives only the diagnostic check (Section 6), where a value with no placement consequence is harmless.

9.4 A body-frame integrator reuses the same resolver

The final piece of the 6-DOF path requires no change to the placement subsystem at all. A future body-frame integrator — one that integrates the rocket’s attitude as well as its translation — supplies the rocket’s world orientation by multiplying the `rootPose` passed to `resolvePlacements(root, rootPose)` by the integrated attitude quaternion. Today `rootPose` is conventionally the identity (the rocket’s aft datum at the world origin, $+z = \text{forward}$); a body-frame integrator passes the live attitude instead.

Because `compose` (Listing 17) already propagates that root orientation down the entire tree, the very same resolver then yields every part’s full world pose — position and orientation — which the force and aero paths can consume to apply thrust and aerodynamic loads in world coordinates. There is no schema change, no resolver-signature change, and no second code path: the single resolver that serves the simulator and the visualizer in 3-DOF (Section 4) serves the attitude-aware integrator unchanged. The 6-DOF work is thereby reduced to three localized edits — stop forcing `childRot` to identity at authoring and load time, rotate the child tensor by the one line of Listing 18, and feed the integrated attitude into `rootPose` — against a data model and a resolver that were built to receive them.

10 Worked example: xl75_multi

This section grounds the abstract machinery of [Sections 3, 4 and 6](#) in a single concrete design from the test corpus, `tests/data/designs/xl75_multi.qrd`. The fixture is small—a nose cone, one body tube, a fin set, and a coupler—yet it exercises all three seat kinds and, critically, it is the design whose legacy `<offset>` encoding silently produced an interpenetrating coupler. Tracing it end to end shows both how the station-pair model expresses each physical relationship with the minimum of authored numbers and how the same model converts the historical silent corruption into a hard, located failure. Every dimension quoted below is read verbatim from the current fixture (it is still a version 0.1 file carrying the legacy CM-to-CM `<offset>` elements); the resolved geometry is the result the station-pair resolver of [Section 4](#) would produce after migration.

The root part and its frame

The root is a solid `ConicalNoseCone` named `MultiNose` with $L = 0.30$ m and $\text{baseRadius} = 0.0395$ m. In the settled longitudinal convention ($+z = \text{forward}$, see [Section 2](#)) the root is visited at the identity pose, so its aft plane—the base rim—sits at world $z = 0$ and the tip is at world $z = +0.30$. Two geometry-derived landmarks anchor everything downstream:

- the aft station, $\{z = 0, r_{\text{outer}} = 0.0395\}$, and the tip station, $\{z = 0.30, r_{\text{outer}} \approx 0\}$;
- the closed-form outer skin, $\text{radiusOuterAt}(z) = 0.0395 (1 - z/0.30)$, which the solid-host rule of [Section 3](#) treats as the host capacity because a solid cone has no bore ($\text{innerCapacityAt}(z) = \text{radiusOuterAt}(z)$).

The cone is also the design’s single frame collision. Its centroidal tensor is computed in a mid-length-origin, base-at- $+z$ frame by `model/parts/ConicalNoseCone.cpp:49`, whose `coneCmOffset` returns $z = L/2 - \bar{h}$ with $\bar{h} = L/4$ for a solid cone. That internal $+z$ is the *opposite* of the chosen forward direction. The resolver does not flip the (correct, tested) tensor math; it applies the explicit forward-frame adapter of [Section 3](#), under which the solid cone’s CM lies $\bar{h}$ forward of the base—that is, $L/4 = 0.075$ m forward of the base (equivalently $3L/4$ aft of the tip), at world $z = 0.075$. This single scalar, `cmStationLocal = 0.075`, is the only consumer of `model/parts/Part.h:96 (getCenterMassOffset)` in the whole design, and it is the reason CM can fall out as a derived quantity rather than being authored.

The four relationships

(1) Body abuts nose — zero authored numbers. The body is a `BodyTube` (inner 0.0376, outer 0.0395, length 0.90). It is attached with the defaulted link, **Abut**, `parentStation01 = 0`, `childStation01 = 1`, `gap = 0`: the child’s fore plane meets the parent’s aft plane. The resolver reads $p = \text{nose.stationAt}(0) = \{z = 0, r_{\text{outer}} = 0.0395\}$ and $c = \text{body.stationAt}(1) = \{z = 0.90, r_{\text{outer}} = 0.0395\}$, giving a child aft-plane origin of $z = 0 + 0 - 0.90 = -0.90$. The body therefore spans world $z \in [-0.90, 0]$, extending *aft* ($-z$)—physically correct, the airframe hangs below the nose. The Layer-1 seam check is clean: $|0.0395 - 0.0395| = 0$.

(2) Fin set seats on the body wall. The `FinSet` (root chord 0.10, $\text{bodyRadius} = 0.0395$, six fins) is attached **OnSurface** at `parentStation01 = 0.06`, `childStation01 = 0`, `gap = 0`. The resolver reads $p = \text{body.stationAt}(0.06)$, i.e. local $z = 0.054$, world $z \approx -0.846$, with $r_{\text{outer}} = 0.0395$, and seats the fin root’s aft end there. Layer 1 confirms radial compatibility: the fin’s `bodyRadius` of 0.0395 equals the tube outer radius within tolerance. The true off-axis radial seat is deferred to the 6-DOF work

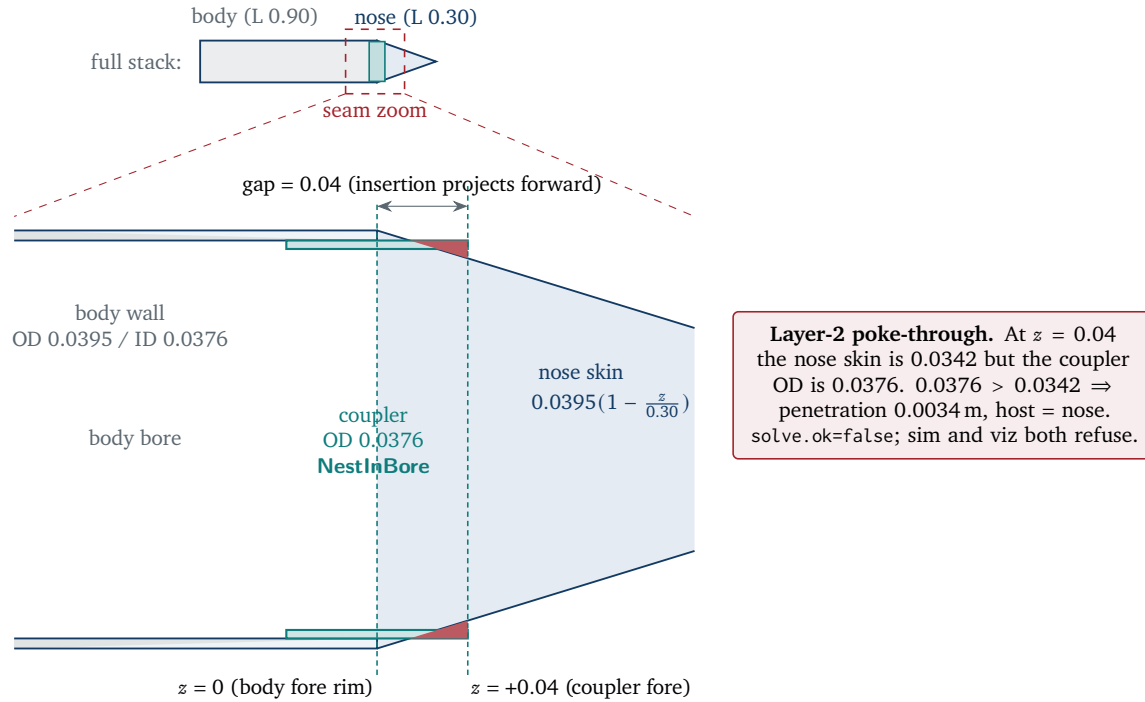


Figure 9. The resolved `xl75_multi` stack in the $+z = \text{forward}$ root frame. The nose occupies $z \in [0, 0.30]$; the body hangs aft over $z \in [-0.90, 0]$; the fin set seats on the body wall near the aft end; the coupler nests into the body fore bore and projects 0.04 m forward of the bore mouth, where its outer radius exceeds the tapering nose skin.

(Sections 4 and 9); because the fin set is axisymmetric its mass sits on the longitudinal axis, so its 3-DOF placement is already exact.

(3) Coupler nests into the body fore bore. The coupler is a short `BodyTube` (inner 0.036, outer 0.0376, length 0.08). Its physical intent is “insert its aft end 40 mm into the body’s fore bore”, expressed as `NestInBore`, `parentStation01 = 1.0`, `childStation01 = 0.0`, `gap = 0.04`: the coupler’s aft plane is the inserted reference. The seat kind makes the gap an insertion depth, so `signedGap = -0.04` (the child moves aft, $-z$, into the bore). The resolver reads $p = \text{body.stationAt}(1.0)$ at body-local $z = 0.90$ (the fore bore mouth, world $z = 0$) and $c = \text{coupler.stationAt}(0)$ at local $z = 0$, giving a child aft-plane origin of $0.90 + (-0.04) - 0 = 0.86$ in the body frame, which composes with the body’s world origin -0.90 to world $z = -0.04$. The coupler thus spans world $z \in [-0.04, +0.04]$: 0.04 m of it is correctly buried in the bore, but its fore 0.04 m projects *forward, past the body fore rim, into the nose region*, because the 0.08 m coupler is longer than its 0.04 m insertion depth. Layer 1 passes—coupler outer radius $0.0376 \leq \text{body inner radius } 0.0376$, so it fits the bore radially—which is precisely why a radial seam check alone is insufficient and the envelope sweep of Section 6 is required.

(4) The forward poke-through, prevented. The coupler’s forward 0.04 m occupies world $z \in [0, +0.04]$, beyond the body and inside the nose’s axial span. The Layer-2 envelope sweep resolves the host by the sorted-interval query at each feature breakpoint:

- Over $z \in [-0.04, 0]$ the host is the body bore, with capacity $r_{\text{inner}} = 0.0376$; the coupler’s $0.0376 \leq 0.0376$ holds, so this span is clean.

- Over $z \in [0, +0.04]$ the body is gone and the only covering envelope is the solid nose cone. The solid-host rule compares the coupler outer radius against the cone *skin*: at the worst station $z = 0.04$, $\text{radiusOuterAt}(0.04) = 0.0395 (1 - 0.04/0.30) = 0.0342$, while the coupler outer radius is 0.0376. Since $0.0376 > 0.0342$, the coupler penetrates the cone skin by 0.0034 m.

Located poke-through: coupler through nose skin

The sweep emits `OverlapDiagnostic{offender = coupler, host = nose, zWorld = 0.04, penetration = 0.0034}` with the message “coupler OD 0.0376 exceeds nose skin radius 0.0342 at $z = 0.04$ forward of the body rim (poke-through 0.0034m)”, and sets `SolveResult.ok = false`. Per the gate in [Section 6](#), *both* consumers refuse the solve: `computeCompositeAt` throws/logs rather than integrate a self-intersecting body, and `RocketMesh` renders the offender in the error colour and surfaces the diagnostic rather than drawing a coupler buried in the cone. The failure is hard and located—never silent. **FAIL**

Table 2. The four `xl75_multi` relationships, their station-pair links, and the resolved result in the $+z = \text{forward}$ root frame.

Relationship	Link (seat; stations; gap)	Result
Body $\rightarrow$ nose	Abut ; 0 $\rightarrow$ 1; gap 0	Body origin $z = -0.90$; spans $[-0.90, 0]$; seam $ 0.0395 - 0.0395 = 0$ OK
Fins $\rightarrow$ body wall	OnSurface ; 0.06 $\rightarrow$ 0; gap 0	Root seats at world $z \approx -0.846$; 0.0395 = 0.0395 OK
Coupler $\rightarrow$ body bore	NestInBore ; 1 $\rightarrow$ 0; gap 0.04	Origin $z = -0.04$; spans $[-0.04, +0.04]$; OD 0.0376 $\leq$ ID 0.0376 OK (Layer 1)
Coupler vs. nose skin	(Layer-2 sweep over $[0, +0.04]$)	At $z = 0.04$, OD 0.0376 $>$ skin 0.0342; penetration 0.0034 FAIL

[Table 2](#) and [Fig. 9](#) together summarize the resolve: three relationships that pass and one that is caught.

Why the legacy file failed silently

In the version 0.1 fixture the coupler is encoded as `<offset z="0.49">` relative to the body—a CM-to-CM displacement in the same $+z = \text{forward}$ frame. That offset slid the coupler bodily *forward* through the cone, and nothing checked it. Worse, the two consumers disagreed on *where* they drew it. The simulator accumulated the raw axial offset, threading `axialStation + pos.z()` through `model/parts/Part.cpp:224` in `accumulateAeroAt` and shifting child tensors to the composite CM in `computeCompositeAt` (`model/parts/Part.cpp:169`). The visualizer walked the tree with `station + offset` at `visualizer/RocketMesh.cpp:472`, translating each *geometric-centered* primitive (the destructure at `visualizer/RocketMesh.cpp:468`). Because a solid cone’s geometric center and its CM differ by exactly the $L/4$ offset of `model/parts/ConicalNoseCone.cpp:49`, the two renderings diverged by that cone offset *on top of* the shared interpenetration error. There was no signal at all: both pictures looked plausible, and a self-intersecting solid was fed to the integrator.

Key correctness detail

The station-pair model removes both faults at once. The stored intent is now “nest 40 mm into the body’s fore bore”, and **NestInBore** fixes the insertion *direction*, so the coupler can no longer be slid arbitrarily forward by an authored number. A single resolver then produces one geometric placement that both consumers read, eliminating the CM-versus-geometric-center divergence by construction. What remains of the original mistake—the 0.04 m of coupler that still projects past the bore mouth—is caught by the envelope sweep with the correct solid-host comparison and reported as a located failure. Resolving the fixture cleanly is then a matter of physical intent: reduce the insertion gap or shorten the coupler until its forward projection no longer exceeds the nose skin.

11 Deliberately deferred / not adopted

A design is defined as much by what it refuses as by what it adopts. The station-pair backbone of [Section 3](#) was selected from five competing sketches and hardened against an adversarial critique; that process produced a set of alternatives that were explicitly considered and set aside. This section records each one together with the reasoning, so that a future contributor revisiting any of these choices inherits the argument rather than rediscovering it. The rejections fall into two kinds: choices that are *wrong* for QtRocket and should stay rejected, and choices that are *correct in the limit* but premature for a rigid inline stack and are therefore deferred rather than dismissed. The distinction matters — a deferred idea is a planned extension point, not a dead end.

+z = aft as the longitudinal convention — rejected (sign-inverted). Adopting +z = aft would invert the longitudinal axis relative to every component that already exists. The visualizer builds its primitives with the fore extent at positive *z* (`buildCone` and `buildTube` place the tip / top at +z), and every legacy `.qrd` fixture — including `tests/data/designs/xl75_multi.qrd:1` — expresses its offsets in a +z = forward frame. Choosing +z = aft would force a whole-axis inversion of the renderer (cap normals, sphere poles, every primitive) *and* a sign-negating migration of every stored file. Adopting +z = forward instead reduces the renderer change to a consumer swap with no sign inversion, and the file migration to a pure re-expression with no axial flip. The convention was settled once, in agreement with the code and the data on disk; see [Section 2](#).

A stored resolved transform as ground truth — rejected (reintroduces staleness). One tempting simplification is to resolve each part’s absolute pose once and store it as the canonical placement. This was rejected because it reintroduces a staleness class that is the exact inverse of the bug this design exists to remove: edit a part’s length and its stored transform is silently wrong, with nothing to recompute it. QtRocket already commits to a “derive, don’t store” discipline for composite quantities, and this design extends that discipline to placement. The `Pose` / `compose` mechanics and the snap-operator verbs are adopted from the scene-graph sketch, but only *intent* is stored; the absolute pose is re-derived on every read.

A rich named-datum enum with an escape hatch — rejected (over-engineering). A datum vocabulary such as `enum { FORE, MID, AFT, ... }` with a parametric escape hatch for arbitrary positions was rejected as paying two costs at once: the rigidity of a closed enum *and* the complexity of the mechanism for escaping it. The named edges are merely fractions — `FORE`, `MID`, and `AFT` are exactly 1, 0.5, and 0 — and an arbitrary station such as a mid-tube rail-button location is just another fraction in $[0, 1]$. Two fractional stations plus a small closed `SeatKind` therefore express everything the enum would, with nothing to extend ([Section 3](#)).

Parallel `childParts` and `childLinks` vectors — rejected (desync hazard). Storing the child

pointers and their links in two side-by-side vectors was rejected because the two arrays must be kept index-aligned by hand across every structural mutation. The single-vector storage at `model/parts/Part.h:326` is touched by `clone()`, `removeChildById`, and `addChildPart`; splitting it invites an off-by-one or a missed erase that silently attaches the wrong link to the wrong child. The design keeps a single vector and changes only the element type, from a `std::tuple<std::shared_ptr<Part>, Vector3>` to a `std::pair<std::shared_ptr<Part>, StationLink>` (Section 3), so the pointer and its link cannot drift apart.

“Every link mandatory” — rejected (punishes the trivial stack). Requiring an explicit link on every attachment was rejected because it taxes the common case to no benefit. The overwhelmingly typical relationship — a child abutting aft of its parent in a straight inline stack — should cost zero authored numbers. The defaulted **Abut** link (parent aft plane to child fore plane, zero gap) preserves that, so `nose->addChildPart(body)` remains a valid, fully specified attachment; see Section 5.

Scalar `radiusOuterAt` for `FinSet` — rejected (dishonest). A fin set is not axisymmetric, so reporting a single scalar outer radius for the overlap sweep would be a fabrication: a `bodyRadius + span disc` would collide falsely with every adjacent body tube. The design instead has a fin set contribute only its *body disc* to the axial envelope, with true azimuthal (sectored) fin interference scoped out of v1 and documented as unchecked, pending a fin-clocking model QtRocket does not yet have. This is honest under-reporting rather than confident wrong-reporting; the diagnostics treatment is detailed in Section 6.

Off-axis ($r > 0$) placement in 3-DOF — deferred to 6-DOF. Placing a part radially off the longitudinal axis — a single rail button, an asymmetric pod — is deferred because seating mass at $x = r$ without an accompanying orientation is itself a half-correct stored placement, the very error class this design eliminates. The radial value r is therefore used *now* only for the seam and overlap *check*, while the pose solve sets $x = y = 0$; correct off-axis seating arrives together with `childRot` and a per-part body-frame CM offset in the 6-DOF work (Section 9).

Three of these warrant fuller treatment, because each guards a correctness invariant that is easy to violate under future pressure to “simplify.”

Storing the resolved transform

The single most attractive — and most dangerous — shortcut is to cache the absolute *Pose* of each part as its source of truth. It would make reads trivially cheap and the data model flat. It is rejected because it merely relocates the staleness bug from the input side to the output side: today a stale *CM-to-CM offset* disagrees with geometry when a length is edited; a stored *resolved transform* would disagree with geometry in exactly the same situations, with no mechanism short of a full re-resolve to detect it. The design keeps placement strictly derived and gates the re-derive on a structural-dirty flag (Section 4), so the cache is an optimization that can never be wrong, not a second copy of the truth that can.

A rich named-datum enum with an escape hatch

A vocabulary of named datums looks more expressive than two bare fractions, but it is strictly worse. It hard-codes the three edges (*FORE/MID/AFT*) that are already representable as $1/0.5/0$, and then needs a parametric escape hatch for every station that is not one of those edges — so it pays for a closed enum and for the machinery to open it. The chosen representation, two

stations in $[0, 1]$ plus a closed three-value `SeatKind`, is both smaller and more general: there is no enum to extend because there is nothing the enum was buying. `SeatKind` itself stays closed deliberately, because a rigid axisymmetric stack genuinely expresses only three radial relationships (Section 3).

The most consequential “not yet” is the full joint graph. Unlike the rejections above, it is the correct long-term model and is deferred only because the present scope does not need it.

A full joint graph with a sibling `<joints>` block

A general design would model attachments as a graph of joint entities, serialized in a sibling `<joints>` block keyed independently of the part tree. This is the correct long-term chassis: clusters, struts, off-axis pods, and the 6-DOF companion-concentricity constraint that Section 6’s Layer 3 will eventually need all require a part to relate to more than its single tree parent — relationships an ownership tree cannot express. It is nonetheless deferred, for two reasons. First, for a rigid inline stack it is over-engineering: every relationship QtRocket authors today is a single parent-to-child seam, which the inline `std::pair<std::shared_ptr<Part>, StationLink>` captures exactly. Second, a sibling joints block keyed by name is an active hazard given that part names need not be unique (`model/parts/Part.h:193`); keeping the link *inline* with its part sidesteps that ambiguity entirely (Section 7). The inline pair is forward-compatible with the graph: when real multi-host designs appear, the single `StationLink` per child can widen into a list of links — or a separate joint table — without disturbing the resolver’s interface or the serialization of the common case.

Deferred is not dismissed

The joint graph and off-axis seating are not rejected ideas; they are staged ones. The data model is deliberately shaped so that adopting them later is additive: `Pose` already carries a `Quaternion` and `compose()` already rotates child translations correctly (the “no rotation” limit at `model/parts/Part.h:43` is a property of today’s identity `childRot`, not of the resolver), and the single-link-per-child storage can widen to a graph. Each deferral therefore leaves a defined extension point rather than a wall, which is why the convention, the seat semantics, and the `Pose` mechanics were all settled now even though only their 3-DOF subset is exercised today.

12 Files this design touches

This section enumerates the concrete edits the refactor makes to the working tree, mapping each file to its change and then bounding the change’s reach. It serves two purposes: it is the implementer’s manifest, and it makes the surface area of the change auditable up front. Every line number below was verified against the current tree (branch `PartPlacement`, namespace `model::part`); the design specification’s own citations were pinned to an older commit and several have since drifted, so the table reflects the live source, not the spec. The defining property of the design is that it is *narrow*: a single in-slot storage swap, one new header, and a fixed set of consumer rewrites. There is no change to the force path, the integrator, or the state vector.

Table 3 is the file-by-file change map. The intent of presenting it this way is to make explicit that almost every edit is either (a) adding a closed-form geometry virtual to a part that already stores the backing radii, or (b) swapping a consumer from the legacy CM-to-CM offset to the resolver’s geometric `Pose`. The single genuinely new artifact is `model/parts/Placement.h` (Section 3); everything

else is an amendment to existing code.

The tuple-to-pair destructure sites in `Part.cpp`

Changing the element type of `childParts` mechanically invalidates every structured binding that walks it. These are local, internal sites — all within `model/parts/Part.cpp` — and each must be re-spelled to bind a `pair<shared_ptr<Part>, StationLink>` (and, where it matters, to read the resolved geometric station rather than the stored offset). The complete, verified set is: `clone` (`model/parts/Part.cpp:117`, including the `emplace_back` that reconstructs the element at `model/parts/Part.cpp:121`), `getCompositeMass` (`model/parts/Part.cpp:131`), `computeCompositeAt` pass 1 (`model/parts/Part.cpp:179`), `accumulateAeroAt` (`model/parts/Part.cpp:221`), `maxFrontalReferenceArea` (`model/parts/Part.cpp:231`), `findById` (`model/parts/Part.cpp:244`), and both loops in `removeChildById` — the `std::get<0>` direct-child scan (`model/parts/Part.cpp:261`) and the recursive descent (`model/parts/Part.cpp:272`).

Two sites the spec's enumeration mis-states

The design spec lists the destructure sites as lines 179, 197, 221, 231, 244, 261, 272. Verification against the live tree corrects this in two ways. First, line 197 is *not* a `childParts` site: it destructures the local `std::vector<std::pair<Vector3, CompositeProperties>>` accumulator in `computeCompositeAt` pass 2, which is unaffected by the storage swap. Second, the spec omits two real `childParts` sites entirely — `clone` (`model/parts/Part.cpp:117`) and `getCompositeMass` (`model/parts/Part.cpp:131`). Both bind the element shape and so both must be migrated. An implementer working from the spec's list alone would leave `clone` and `getCompositeMass` broken.

Blast radius: the public `getChildParts()` return type

The destructure sites above are private to `Part`. The change with reach beyond the class is the public accessor: `getChildParts` (`model/parts/Part.h:209`) currently returns `const std::vector<std::tuple<std::shared_ptr<Part>, Vector3>>&`, and the refactor changes that to the `pair<ptr, StationLink>` vector. Every external consumer that iterates the returned view therefore breaks at compile time and must be migrated — a `std::get<0>` → `.first` (and `std::get<1>` → `.second`) rename, or the equivalent structured-binding rewrite. This is mechanical and enumerated, not hidden.

The shim does not cover the accessor

The `[[deprecated]]` transitional `addChildPart(child, Vector3)` overload (Section 5) keeps legacy *setter* call sites compiling through the migration. It does **not** shield the *accessor*: `getChildParts`'s element type is part of the public contract, and there is no way to return both shapes from one signature. Accessor consumers must therefore be migrated explicitly, in lockstep with the storage swap, rather than deferred behind the shim.

The confirmed external call sites are the visualizer's tree walk (`visualizer/RocketMesh.cpp:466`, which today reaches the element through `std::get<0>` / `std::get<1>`) and the serializer's child-writing loop (`model/DesignSerializer.cpp:73`); both files already appear in Table 3 for their primary changes, so the accessor migration folds into work those files receive regardless. Beyond these, the CLI and GUI tree-walkers and any tests that iterate `getChildParts` update the same way.

Table 3. Files this design touches, with the change applied to each. Line numbers are pinned to the current working tree, not the design spec. *New* marks the one wholly new file; all other entries amend existing code.

File	Change
model/parts/Part.h:326	Storage member <code>childParts</code> changes type from <code>std::vector<std::tuple<std::shared_ptr<Part>, Vector3>></code> to <code>std::vector<std::pair<std::shared_ptr<Part>, StationLink>></code> — a single in-slot swap (Section 3). Adds the geometry virtuals <code>stationAt</code> , <code>radiusOuterAt</code> , <code>radiusInnerAt</code> , <code>axialLength</code> , <code>isSolid</code> , and <code>cmStationLocal</code> to the base class (none exist today; the only base geometry virtual is <code>getReferenceArea</code> , model/parts/Part.h:176). Promotes the new <code>StationLink</code> <code>addChildPart</code> overload (model/parts/Part.h:239) and updates the <code>getChildParts</code> return type (model/parts/Part.h:212). Adds a <code>placementDirty</code> flag and a <code>resolvedCache</code> member.
model/parts/Placement.h	<i>New</i> . Houses the value types and free functions: <code>Station</code> , <code>StationLink</code> , <code>SeatKind</code> , <code>Pose</code> , <code>Placed</code> , <code>resolvePlacements</code> , <code>sweepOverlaps</code> , <code>OverlapDiagnostic</code> , and <code>SolveResult</code> (Sections 3, 4 and 6).
model/parts/Part.cpp	<code>computeCompositeAt</code> (model/parts/Part.cpp:169) is rewritten to consume the resolver's <code>Pose</code> and derive CM through <code>cmStationLocal</code> , preserving the two-pass mass-weighted CM and parallel-axis structure (Section 4). <code>accumulateAeroAt</code> (model/parts/Part.cpp:214) threads each child's resolved <code>Pose.origin.z()</code> instead of <code>axialStation + pos.z()</code> (model/parts/Part.cpp:224). Adds <code>ensurePlacementCache</code> . Every structured binding over <code>childParts</code> is updated from the tuple to the pair (see below).
model/parts/ ConicalNoseCone.cpp	Adds <code>radiusOuterAt</code> (the linear taper $\text{baseRadius} \cdot (1 - z/L)$), the <code>cmStationLocal</code> frame adapter of Section 3 (the cone is the only part that overrides it), and <code>isSolid</code> . The internal cone CM math in <code>coneCmOffset</code> (model/parts/ConicalNoseCone.cpp:49) is <i>not</i> touched; only the adapter that reconciles its frame is added.
model/parts/BodyTube.h, model/parts/BodyTube.cpp, model/parts/FinSet.h, model/parts/FinSet.cpp, model/parts/HollowSphere.h, model/parts/ HollowSphere.cpp	Each gains <code>radiusOuterAt</code> / <code>radiusInnerAt</code> and <code>isSolid</code> backed by its existing wall radii (BodyTube: model/parts/BodyTube.h:49; HollowSphere: model/parts/HollowSphere.h:52). <code>FinSet</code> reports its <i>body-disc</i> radius only (<code>getBodyRadius</code> , model/parts/FinSet.h:67) for axial occupancy; sectorized fin interference is scoped out (Section 6).
visualizer/RocketMesh.cpp	<code>walk</code> (visualizer/RocketMesh.cpp:427) consumes the resolver's <code>Pose</code> instead of accumulating <code>station + offset</code> (visualizer/RocketMesh.cpp:472). Because the convention is $+z = \text{forward}$ — already the visualizer's own frame (visualizer/RocketMesh.cpp:156) — this is a pure consumer swap with no axis flip (Section 2).
model/DesignSerializer.cpp	<code>writeOffset</code> (model/DesignSerializer.cpp:53) becomes <code>writeLink</code> , emitting <code>seat/parentStation/childStation/gap</code> instead of an <code>x/y/z</code> offset. <code>buildPart</code> (model/DesignSerializer.cpp:117) reads a <code><link></code> element. The version string bumps from 0.1 (model/DesignSerializer.cpp:156) to 0.2; the loader's <code>major == "0"</code> tolerance (model/DesignSerializer.cpp:186) already accepts it, so no gate change is needed (Section 7).

A new base virtual: `getLength()`

One amendment in [Table 3](#) is easy to overlook because it is implied rather than stated as a line edit: `getLength()` is *promoted to a base Part virtual*. Today there is no `getLength()` on `Part` at all. It exists only as a non-virtual accessor on the concrete types that have an axial length — `ConicalNoseCone` (`model/parts/ConicalNoseCone.h:53`) and `BodyTube` (`model/parts/BodyTube.h:51`) — while `FinSet` and `HollowSphere` have none. The base `stationAt` and the default `axialLength()` both need a part-level length to resolve fractional stations, so the refactor lifts `getLength()` onto the base class.

`HollowSphere` consequently gains a `getLength()` override: a sphere has no length member (`model/parts/HollowSphere.h:52` exposes only radii), so its axial extent must be reported as its diameter, $2 \cdot \text{outerRadius}$. Without this override the inherited default `axialLength()` would have nothing to read for a sphere. This is the smallest of the file changes, but it is the one that makes the fractional-station model ([Sections 3 and 4](#)) well defined for every concrete part type rather than only the two that happened to carry a length already.

A Reproducing this document (toolchain)

This whitepaper is itself a small, self-contained build: a contributor who clones the repository should be able to regenerate the PDF, with every diagram, from the sources checked into `docs/PartPlacementDesignLatex/`. This appendix documents that build so that the figures referenced throughout the preceding sections remain reproducible rather than opaque binary artifacts. The guiding principle is that nothing rendered into the final document is committed in its rendered form: the figures are derived, on demand, from text sources by a deterministic pipeline, so a reviewer can audit *how* every diagram was produced and regenerate it.

A.1 Directory layout

The document is assembled from a small number of well-separated source kinds. The master file `PartPlacement.tex` carries only the document scaffolding—title block, front matter, and an ordered list of `\input` directives—while the prose lives in `sections/`, one file per section (this appendix is `sections/A1-reproducibility.tex`). The shared style contract is isolated in `preamble.tex`, which defines the colour palette, the fonts (XCharter text, `newtxmath`, Inconsolata mono), every callout environment, and the domain macros (`\src`, `\classfig`, `\tikzfig`, and the rest) that the section files depend on. Keeping the preamble separate is what lets each section file be authored and reviewed in isolation: a section is just a body, never a standalone $\text{\LaTeX}$ program.

Two diagram toolchains, by figure kind

Class diagrams are authored in PlantUML and live as `diagrams/*.puml`; activity, component, geometry, and worked-example figures are hand-built in TikZ and live as `diagrams/tikz/*.tex`. The split is deliberate: PlantUML excels at laying out box-and-line class structure with no manual coordinate work, whereas the flow, axis, and geometry figures need pixel-level control that is far easier to express directly in TikZ than to coerce out of a layout engine.

The remaining directories are mechanical. `diagrams/out/` holds the rendered diagram artifacts (the intermediate SVGs and the PDFs that $\text{\LaTeX}$ actually includes); it is a build output and is reconstructed from sources. `tools/plantuml.jar` is the pinned PlantUML renderer, fetched on demand rather than vendored as a hand-managed binary. The shared PlantUML theme `diagrams/_style.iuml:1` is given the

`.iuml` extension precisely so that it stays out of the Makefile’s `diagrams/*.puml` glob—it is an include, not a diagram, and must not be rendered as one.

A.2 The rendering pipeline

Diagrams flow through three stages before they reach the document. Each PlantUML source is first rendered to SVG, the SVG is converted to a vector PDF, and that PDF is pulled into the document by `\includegraphics` (via the `\classfig` macro). Schematically:

```
diagrams/*.puml      --(plantuml -tsvg)-->    diagrams/out/*.svg
diagrams/out/*.svg   --(rsvg-convert -f pdf)--> diagrams/out/*.pdf
diagrams/out/*.pdf + sections/*.tex --(latexmk -pdf)--> PartPlacement.pdf
```

SVG is used as the intermediate because it is PlantUML’s cleanest vector output; the SVG-to-PDF step uses `rsvg-convert` so the figures embed as resolution-independent vectors rather than rasterized bitmaps. The converter is configurable—the Makefile exposes a `SVG2PDF` variable (Makefile:33) so a contributor without `rsvg-convert` can substitute, for example, an Inkscape export command without editing the rules. The TikZ figures need no such pipeline: they are $\text{\LaTeX}$ sources included directly by `\tikzfig` and compiled in line by the main engine.

The document engine is `pdflatex`, driven by `latexmk` so that the multi-pass dance (cross-references, the table of contents, `cleveref`, and the bookmark tree) converges automatically. The one non-default requirement is `-shell-escape`: the listings throughout this document are typeset by `minted`, which shells out to the Python `pygmentize` tool to perform syntax highlighting. As the preamble notes at `preamble.tex:3`, `minted` cannot run without shell-escape, so the Makefile’s `latexmk` invocation enables it unconditionally (Makefile:35).

PlantUML resolves `!include` relative to the working directory

Every `*.puml` begins with `!include _style.iuml` to pull in the shared theme (`diagrams/class-current.puml:4`). PlantUML resolves that path relative to the *current working directory*, not the file being rendered, so running it from the repository root would fail to find the theme. The Makefile therefore `cds` into `diagrams/` before invoking the renderer, passing the jar and the output directory as absolute paths (Makefile:60). This is the single most common foot-gun when rebuilding a diagram by hand: invoke PlantUML from `diagrams/`, or the `!include` will not resolve.

A.3 Authoring format: a sample class diagram

A PlantUML class diagram is a compact text description of types and their relations; the renderer handles all layout. Listing 19 shows the head of `diagrams/class-current.puml` (the source behind Fig. 1): it includes the shared theme, sets a title, and declares the abstract `Part` with its members and the stereotype that drives its colour. The full file goes on to declare the concrete part subclasses, the composition edge, and an explanatory note, but the excerpt is enough to show that the diagram is fully auditable text—there is no hand-placed geometry to drift out of sync with the code it depicts.

The shared theme `diagrams/_style.iuml:21` maps stereotypes (`«abstract»`, `«concrete»`, `«pod»`, `«newtype»`, `«enumc»`, `«freefn»`, `«legacy»`) onto the same colour palette as the body text, so the class diagrams and the prose share one visual language. Per its own header comment, this PlantUML build rejects the block form `skinparam class«stereo» { }` (it exits with status 200); the theme therefore uses the supported `Property«stereo»` suffix form inside a single `skinparam class { }` block (`diagrams/_style.iuml:5`). A contributor adapting the theme should preserve that form.

```

@startuml class-current
!include _style.iuml
title Current model::part composite tree - CM-to-CM child offset

abstract class "Part" as Part <<abstract>> {
    +using Id = std::uint64_t
    --
    -cm : Vector3
    -needsRecomputing : bool
    -childParts : vector<tuple<shared_ptr<Part>, Vector3>>
    --
    +getId() : Id
    +addChildPart(child, Vector3 position)
    +getChildParts() : const vector<tuple<..>>&
    #computeCompositeAt(t)
}
Part "1" o-- "childParts *" Part : composite (by move)
@enduml

```

Listing 19. Head of `diagrams/class-current.puml`: a PlantUML class diagram is plain text, themed by `!include _style.iuml` and rendered to SVG then PDF by the Makefile.

A.4 Commands

The Makefile wires the whole pipeline into a handful of phony targets. From `docs/PartPlacementDesignLatex/`:

```

make          # default: render all diagrams, then build PartPlacement.pdf
make diagrams # render only the PlantUML class diagrams to diagrams/out/*.pdf
make tools    # fetch tools/plantuml.jar (pinned version) if it is missing
make clean    # remove LaTeX aux files, the minted cache, and diagram SVGs
make veryclean # clean, and also remove PartPlacement.pdf + rendered diagram PDFs

```

The dependency graph is arranged so that a clean checkout builds in a single `make` invocation: the PDF target depends on the rendered diagram PDFs ([Makefile:67](#)), each diagram PDF depends on its SVG, and each SVG depends on its `*.puml` source, the shared theme, and—as an order-only prerequisite—the PlantUML jar. The jar rule ([Makefile:50](#)) fetches a pinned PlantUML version over `curl` into `tools/` only when the jar is absent, so the network is touched at most once and the version is reproducible. The intermediate SVGs are marked `.SECONDARY` ([Makefile:42](#)) so `make` does not delete them as transient by-products between the two diagram stages.

`make clean` runs `latexmk -c` to drop the $\text{\LaTeX}$ auxiliary files, then removes the `minted` cache (the `_minted-*` directories that hold the cached highlighter output) and the intermediate SVGs, while leaving the finished PDF and the rendered diagram PDFs in place for inspection; `make veryclean` additionally runs `latexmk -C` and deletes the final PDF and the entire `diagrams/out/` tree, returning the directory to a pristine, source-only state.

A.5 Prerequisites

To rebuild from a clean checkout a contributor needs: a Java runtime (`java`, to run the fetched `tools/plantuml.jar`); `curl` (to fetch the jar on first build); `rsvg-convert` or a substitute set via `SVG2PDF` (for the SVG-to-PDF step); and a $\text{\TeX}$ distribution providing `latexmk`, `pdflatex`, and the `minted` package together with its `pygmentize` helper. With those present, `make` reproduces this document end to end.

B Glossary

This appendix defines the domain terms as they are used throughout this whitepaper. Terms appear in alphabetical order. Where a term names a concrete construct, the relevant source location is cited against the current working tree; where a term names a type or routine introduced by this design, it is identified as new and unimplemented.

Abut A **Abut** `SeatKind`: a rim-to-rim seat between two parts of equal radius (a tube fore rim meeting a tube aft rim). The radial check requires the two stations' outer radii to match within tolerance, and the gap is interpreted as a forward standoff along $+z$ = forward, flush when the gap is zero. It is the default seat, so the trivial stack costs no authored numbers.

CM-to-CM legacy link The current attachment representation: each child stores a `Vector3` giving the child's center of mass relative to the parent's center of mass, in `model/parts/Part.h:326`. Because a part's CM is not in general at its mid-length (a solid cone's CM sits $L/4$ from the base), this couples placement to a derived quantity and forces authors to hand-compute offsets. This design replaces it with a geometric `StationLink`.

Composite CM / composite inertia The center of mass and inertia tensor of a part together with all its descendants. They are cached on each part (`compositeCm` at `model/parts/Part.h:320`, `compositeInertiaTensor` at `model/parts/Part.h:300`) and recomputed by the two-pass routine `computeCompositeAt` (`model/parts/Part.cpp:169`): pass one builds the mass-weighted composite CM, pass two shifts every child tensor to that CM via the parallel-axis theorem. The composite tensor is full mass-weighted ($\text{kg} \cdot \text{m}^2$), whereas a part's own `inertiaTensor` is stored per unit mass (m^2).

Cone CM-frame adapter The single explicitly stated transform that reconciles the cone's internal CM frame with the resolver's frame. `coneCmOffset` (`model/parts/ConicalNoseCone.cpp:49`) expresses the cone CM in a mid-length-origin, base-at- $+z$ frame, whose $+z$ is the opposite of the chosen forward. The resolver instead works aft-plane-origin, $+z$ = forward, where the forward-frame CM coordinate is $cmZ_{\text{fwd}} = \bar{h}$ ($L/4$ solid, $L/3$ shell — the CM sits near the wide base). The cone is the only part whose `cmStationLocal` override bridges a frame flip.

cmStationLocal A new per-type `Part` virtual returning a part's own CM as a single scalar axial coordinate in its $+z$ = forward local frame. Symmetric parts return $L/2$; the cone overrides it with the adapter above. It is the sole consumer of `getCenterMassOffset`, so no implicit cross-frame vector addition survives.

Envelope sweep Layer 2 of the diagnostics (`sweepOverlaps`, new): after the whole tree resolves, build world axial intervals for every part, sort by aft station ($O(N \log N)$), then sample each offender at feature breakpoints and test its outer radius against each candidate host's capacity using the solid-host rule. It runs once per structural change, never per integration step.

getCenterMassOffset The accessor returning a part's CM relative to the component middle (`model/parts/Part.h:96`), zero for symmetric parts and nonzero for a cone. It is documented but unconsumed today; this design finally consumes it, exclusively inside `cmStationLocal`.

hostCapacity The radius a host occupies at a given station, defined by the solid-host rule as `radiusInnerAt(z)` for a bored part and `radiusOuterAt(z)` for a solid part. An offender of outer radius r_{off} fits iff $r_{\text{off}} \leq \text{hostCapacity}(z) + \text{tol}$.

innerCapacityAt The new non-virtual `Part` helper that evaluates `hostCapacity` for a part at a local station: the bore for a tube, the outer skin for a solid. One predicate, correct for both host kinds, used by the envelope sweep.

NestInBore A **NestInBore** `SeatKind`: the child's outer radius seats inside the parent's bore. The radial

check requires child OD within parent ID, and the gap is an insertion depth ≥ 0 that drives the child *aft* ($-z$) into the bore, so the signed gap is negated relative to **Abut**.

OnSurface An **OnSurface** **SeatKind**: the child seats radially on the parent's outer wall at a chosen station (a fin root on a body tube). The radial check compares the child's radius against the parent's outer radius at the station, and the gap is an axial standoff along the wall.

OverlapDiagnostic A new POD recording one located interference: the offender and host **Part::Ids**, the world station of the worst violation, the penetration depth in metres, and a human-readable message. **Part::Id** is `std::uint64_t` (`model/parts/Part.h:56`).

Parallel-axis theorem (parallelAxisTerm) The displacement tensor $f(d) = (d \cdot d)I_3 - d d^T$ that, scaled by a body's mass and added to its centroidal tensor, shifts that tensor to a parallel axis offset by d (`model/parts/Part.cpp:17`). The composite inertia pass applies it to every child; this design preserves the math and changes only the input d from CM-to-CM offsets to resolver-derived geometric stations.

Part composite tree The ownership tree of rocket components rooted at **Part** (`model/parts/Part.h`, namespace `model::part`). Each node is both a single component and the composite of itself with its children; children are stored in `childParts` (`model/parts/Part.h:326`) and adopted by move via `addChildPart` (`model/parts/Part.h:239`). `clone()` deep-copies and re-mints ids.

placementDirty vs. the mass-delta gate Two distinct cache gates. The existing mass-delta gate, `ensureCompositeCache` keyed on `builtAtCompositeMass` (`model/parts/Part.h:288`), fires on every integration step during a burn because the composite mass changes each step. The new `placementDirty` flag would instead gate the resolver cache on *structural* change only (a part added, removed, or geometrically edited). Today only the single `needsRecomputing` flag (`model/parts/Part.h:322`) exists, propagated upward by `markAsNeedsRecomputing` (`model/parts/Part.h:293`); the separate `placementDirty` and `resolvedCache` are added by this design.

Placed A new value pairing a resolved **Part** pointer with its **Pose**. The resolver returns a `std::vector<Placed>` in deterministic depth-first attachment order.

Pose A new value type carrying an origin (the part-local aft-plane origin in the root frame) and an orientation quaternion (identity in 3-DOF). Its `compose` operator chains a child pose through its parent, rotating the child translation, so the same routine serves 6-DOF unchanged.

Propagatable The model $\leftrightarrow$ sim bridge interface implemented by **RocketModel**, exposing `getForces`, `getTorques`, `getMass(t)`, and `terminateCondition` to the propagator. It is unaffected by this design, which changes only how composite quantities are derived, not how they are consumed by the integrator.

Resolver (resolvePlacements) The new single authority for absolute placement: a depth-first walk over the ownership tree from a root pose, producing a **Placed** for every part by pure geometry (no CM, no time). Both the simulator (`computeCompositeAt`, `accumulateAeroAt`) and the visualizer (`RocketMesh::walk`) consume its output, so the two cannot disagree.

SeatKind A new closed enum of three radial seat relationships — **Abut**, **NestInBore**, **OnSurface** — that governs the radial compatibility check and the sign of the gap. It is intentionally closed because a rigid axisymmetric stack expresses only these three relationships.

SolveResult A new aggregate holding an *ok* flag and the list of **OverlapDiagnostics** from a resolve. When *ok* is false, both the simulator and the visualizer refuse the solve; it is produced once per structural resolve and cached, so the gate costs nothing per integration step.

Solid-host rule The diagnostic predicate that resolves the ambiguity of a solid part, which has no bore and would otherwise flag everything. A host occupies $[0, \text{hostCapacity}(z)]$, where capacity is the bore for a bored part and the outer skin for a solid; an offender fits iff its outer

radius does not exceed that capacity within tolerance. This makes the coupler-versus-cone test the correct poke-through check.

Station landmark The new `Station` value $\{z, r_{\text{outer}}, r_{\text{inner}}\}$ returned by `stationAt` for a fractional station: the live axial coordinate and radii in the part's $+z = \text{forward}$ local frame, derived from geometry and never stored.

Station / fractional station (`station01`) A position along a part's longitudinal axis expressed as a fraction in $[0, 1]$, where 0 is the aft plane and 1 the fore plane. The absolute axial coordinate is `station01 × getLength()`, recomputed on each read so that editing a length moves the seam. `getLength()` is promoted to a base `Part` virtual by this design; today it exists only on `ConicalNoseCone` and `BodyTube`.

StationLink The new stored attachment value replacing the CM-to-CM `Vector3`: a parent fractional station, a child fractional station, an axial gap, a `SeatKind`, and a per-seam child rotation (identity in 3-DOF). It encodes physical intent — which feature meets which — rather than a derived coordinate, so the storage type becomes `std::vector<std::pair<std::shared_ptr<Part>, StationLink>>`.

3-DOF vs. 6-DOF The simulator currently integrates only the three translational degrees of freedom; orientation is carried in `StateData` but not yet integrated. `Part` documents the rigid no-relative-rotation assumption (`model/parts/Part.h:43`). This design shapes in the 6-DOF path at zero schema cost — `Pose` carries an orientation, `StationLink` carries a per-seam rotation, and one composition line would rotate each child tensor — while remaining bit-identical to today in 3-DOF.

$+z = \text{forward}$ convention ($+z = \text{forward}$) The settled longitudinal convention: $+z$ points toward the nose tip, every part's local origin sits on the axis at its aft plane, and a part occupies $z \in [0, +\text{length}]$. It matches the visualizer and the existing `.qrd` files, so the renderer change is a consumer swap and the file migration a re-expression, both with no axial sign flip. $+z = \text{aft}$ was rejected for requiring exactly those inversions.